SIAM J. MATRIX ANAL. APPL.
Vol. 11, No. 1, pp. 134–172, January 1990
© 1990 Society for Industrial and Applied Mathematics
010

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

THE ROLE OF ELIMINATION TREES IN SPARSE FACTORIZATION*

JOSEPH W.H. LIU†

Abstract. In this paper, the role of elimination trees in the direct solution of large sparse linear systems is examined. The notion of elimination trees is described and its relation to sparse Cholesky factorization is discussed. The use of elimination trees in the various phases of direct factorization are surveyed: in reordering, sparse storage schemes, symbolic factorization, numeric factorization, and different computing environments.

Key words. elimination tree, sparse matrix, Cholesky factor, reordering, symbolic factorization, numeric factorization

AMS(MOS) subject classifications. 65F50, 65F25

1. Introduction.

The elimination tree plays an important role in many aspects of sparse matrix factorization. It provides structural information relevant to the sparse factorization process. The purpose of this paper is to provide a unified study of this important structure and to survey its uses in various phases of sparse factorization.

Throughout this paper, unless otherwise specified, A is assumed to be a large sparse n-by-n symmetric positive-definite matrix. We consider direct methods for the solution of the linear system

$$A\,x \;=\; b\,.$$

The matrix A is factored into LL^T, where L is lower triangular and is the Cholesky factor of A. The solution vector x is then obtained by forward and backward substitution using L. The elimination tree of A is defined using the structure of the Cholesky factor L of A. It can therefore be characterized using only the structure of the given matrix A.

The structure of an elimination tree was used implicitly long before its importance was recognized. The term elimination tree was used by Duff [6], although the actual structure studied by him is slightly different from the one in this paper. Jess and Kees [34] use this term to refer to a tree structure introduced by them for studying parallel elimination. Their structure is, in fact, a special case of the elimination tree studied here. Schreiber [55] is perhaps the first one to formally define the elimination tree structure. In [36], Liu uses the term "elimination tree" to refer to the structure introduced by Schreiber. It is this tree structure that we are going to consider in this paper.

Variants of the basic elimination tree structure appear in the literature under different names. It is used as a *dissection tree* to study nested dissection [18], [31] in the context of optimal sparse matrix reordering. The *element merge tree* introduced by Eisenstat, Schultz, and Sherman [13] in their element model has a structure close to the elimination tree. Duff and Reid [10] use an *assembly tree* to determine the assembly order in the multifrontal method, and its structure is a generalized version

* Received by the editors December 10, 1987; accepted for publication (in revised form) May 10, 1989. This research was partially supported by the Natural Sciences and Engineering Research Council of Canada under grant A5509, by the Applied Mathematical Sciences Research Program, Office of Energy Research, U.S. Department of Energy under contract DE-AC05-84OR21400 with Martin Marietta Energy Systems Inc., and by the U.S. Air Force Office of Scientific Research under contract AFOSR-ISSA-86-00012.

† Department of Computer Science, York University, North York, Ontario, Canada M3J 1P3 (joseph@yuyetti.bitnet or NA.JLIU@na-net.stanford.edu).

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

of the elimination tree. The *row merge tree* used by Liu in [38] can be viewed as an extension of the elimination tree for the study of sparse orthogonal factorization. This paper gives a unified treatment of this structure, which has proved to be a valuable tool in sparse elimination.

The reader is assumed to be familiar with the graph-theoretic terminology used in the study of sparse elimination: fill, ordering and permutation, elimination graph model, chordal graphs, and other related concepts. All the necessary material can be found in [8] and [22]. Moreover, the basic terminology and concepts in the study of trees will be assumed: parent/child, ancestor/descendant, paths, root, subtree, leaf, and others. The reader is referred to [1].

In this survey paper, we use many results that have already appeared in the literature. Such results are quoted and properly referenced, and their proofs are usually omitted. However, results that are generally known among sparse matrix researchers, but have not been formally dealt with, are treated in greater detail. This is to provide a good foundation for future works on elimination trees.

Another objective of this paper is to point out the relevance of the elimination tree in many existing sparse algorithms. The connections may not be stated in previous discussions of these algorithms in the literature. Here, we provide the direct link to this tree structure. The paper also contains some new results on the elimination tree, one of which is its use in the intersection graph representation of chordal graphs in §4.2.

An outline of this paper follows. In §2, the notion of an elimination tree is formally defined in terms of the Cholesky factor L of a sparse matrix A. A simple matrix example is introduced, and it will be used throughout the remainder of this paper. We also offer two new views of the elimination tree: one as the transitive reduction of the directed graph of L, and another as a depth-first search tree of the filled graph.

Section 3 provides properties of the elimination tree that are relevant to the sparse Cholesky factor L. Nonzeros in the factor matrix can be characterized in terms of paths in the elimination tree. Both the row and column structures of L can also be expressed in terms of structures of this tree. Section 4 gives some observations on the use of elimination trees for chordal graphs.

In §5, we consider efficient ways of determining the elimination tree for a given sparse matrix A. An efficient algorithm can be formulated in terms of the basic set union operations, introduced by Tarjan [58]. Some experimental results are provided to offer some practical observations in this regard.

Sections 6–10 examine the role of the elimination tree in various aspects and different phases of sparse factorization. Section 6 considers the use of elimination trees for finding equivalent matrix reorderings. It includes reorderings that preserve the elimination tree structure, and those that completely restructure it. In §7, we study various sparse storage schemes that makes use of elimination tree structures.

Section 8 examines the connection of the elimination tree with the symbolic factorization phase. Existing symbolic factorization algorithms are shown to use this tree structure implicitly. Many numerical factorization schemes also use this tree structure. In §9, we consider its role in the multifrontal method, the minimal storage scheme, the general row merging scheme, and sparse indefinite factorization. In specific computing environments, the elimination tree can also be used to improve on the basic factorization scheme, and we consider it in §10. Finally, §11 contains some remarks on future research directions.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

2. On elimination trees.

2.1. Trees in sparse matrices. Undirected graphs are useful tools in the study of symmetric matrices. See, for example, [8], [22], and [52]. A given symmetric sparse matrix A can be structurally represented by its associated graph $G(A) = (X(A), E(A))$, where nodes in $X(A)$ correspond to rows and columns of the matrix and edges in $E(A)$ correspond to nonzero entries. We shall use $\{u, v\}$ to indicate an edge between two nodes u and v.

In this section, we briefly review properties of sparse matrices whose associated graphs are *trees*, in preparation for the introduction of elimination trees in the next section. Without loss of generality, we assume that the given matrix is irreducible so that its graph is connected. Let A be an n-by-n symmetric positive-definite irreducible matrix and let $G(A)$ be its associated graph, which is a tree. It is well known that such a matrix A is a *perfect elimination matrix*; that is, it has a permutation P such that the permuted matrix PAP^T does not suffer any fill in its Cholesky factorization [52].

Orderings that will produce no extra fill are sometimes referred to as *perfect elimination orderings* [52]. Such orderings for tree structures are easy to obtain. Indeed, the popular *minimum degree ordering* on $G(A)$ will be appropriate. Or, more simply, take any node in the associated tree as the root, any ordering that numbers children nodes before their parent node will introduce no fill. In particular, a postordering [1] of the rooted tree will be one such ordering.

For a given rooted tree, we define a *topological ordering* of the tree to be an ordering that numbers children nodes before their parent node. This is consistent with the notion of topological orderings for directed acyclic graphs (directed graphs without cycles) used in the literature [58]. A topological ordering of a directed acyclic graph is one such that for every directed edge from a node u to v, u is ordered before v. In the case of a rooted tree, if we treat each tree edge as a directed edge that goes from a child to its parent, our definition of a topological ordering of a rooted tree is the same as that used for directed acyclic graphs.

Assume the matrix A (whose associated graph is a tree) has already been ordered by a topological ordering. Let $x_1, x_2, \cdots, x_n$ denote the nodes in the associated graph/tree, where node x_j corresponds to the jth row/column of the matrix A. Note in the topological ordering, except the root, each node at its elimination is connected to only one uneliminated node (namely, its parent). It follows then that in the lower triangular part of A, each column has exactly *one* off-diagonal nonzero except the last column.

Such matrix structure can be represented by the subscripts of the off-diagonal nonzeros in the columns. For each column $j < n$ of A, let

$$PARENT[j] = p, \text{ where } a_{pj} \neq 0 \text{ and } p > j.$$

For completeness, we let $PARENT[n] = 0$. The function $PARENT[*]$ uniquely characterizes the associated tree $G(A)$ of A. If we consider the tree $G(A)$ as rooted at x_n, for each node x_j other than the root, its parent node in the tree is given by $x_{PARENT[j]}$.

2.2. Notion of elimination trees. Trees form a class of data structure that is easy to store and manipulate. However, in practice, it is rare that sparse matrices have associated graphs in the form of trees. In this section, for a general symmetric positive-definite sparse matrix, we introduce a tree structure that is useful in the study of sparse factorization. It may be viewed as a generalization of the tree in §2.1.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Let A be a given n-by-n sparse symmetric positive-definite irreducible matrix. Consider its Cholesky factorization $A = LL^T$. As before, let $G(A)$ be the undirected graph associated with A, and let $x_1, x_2, \cdots, x_n$ be the sequence of nodes. Moreover, let $G(F)$ be the associated filled graph, where $F = L + L^T$ is the filled matrix of A. It is well known that $G(F)$ has the same set of nodes and is a supergraph of $G(A)$.

Consider the Cholesky factor L and the filled matrix F of A. Since the matrix A is irreducible, it can be readily verified that each of the first $n-1$ columns of L has at least one off-diagonal nonzero. For each column $j < n$ of L, remove all the nonzeros in this column except the first nonzero below the diagonal. Let L_t be the resulting matrix and $F_t = L_t + L_t^T$. The graph $G(F_t)$ is a tree structure, and it depends entirely on the structure of the original sparse matrix A and its initial ordering. We use $T(A)$ to denote this tree structure and refer to it as the *elimination tree* of A.

An example is given in Fig. 2.1, which illustrates the structures of the matrices A, F, and F_t. Each diagonal entry is labeled by the corresponding node in the graph. Off-diagonal nonzeros are indicated by "•" while "∘" is used to denote a fill in the matrix. The corresponding graphs of $G(A)$, $G(F)$, and $T(A)$ are given in Fig. 2.2. A dotted line in $G(F)$ is used to indicate a filled edge in the graph. Note that some of the tree edges in $T(A)$ are filled edges of $G(F)$ (for example, the tree edge between nodes g and h). This same example will be used throughout the remainder of this paper.

The elimination tree $T(A)$ has the same node set as $G(A)$ and is a *spanning tree* [1] of the filled graph $G(F)$ of A. We define the node x_n to be the root of this tree $T(A)$. The elimination tree structure can be conveniently represented by the $PARENT[*]$ vector of F_t. In terms of the triangular factor L, we have, for $j < n$,

$$PARENT[j] = \min \{ i > j \mid \ell_{ij} \neq 0 \}.$$

It should be pointed out that this is a generalization of the $PARENT[*]$ vector introduced in §2.1; since when $G(A)$ is a tree and is ordered with no fill, we have $a_{ij} \neq 0$ if and only if $\ell_{ij} \neq 0$. Indeed, in this case, $G(A) = G(F) = T(A)$.

In general, $G(A)$ is a subgraph of $G(F)$ due to fills. However, the two elimination trees $T(A)$ and $T(F)$ are identical (since F is a perfect elimination matrix with no additional fill). Often, we simply use T to refer to this tree. The next result contains another simple property of the elimination tree. It follows directly from the definition of the $PARENT$ vector.

PROPOSITION 2.1. *If x_i is a proper ancestor node of x_j in the elimination tree, then $i > j$.* □

To facilitate discussions in subsequent sections, we introduce the subtree notation. We use $T[x]$ to represent the subtree of $T(A)$ rooted at the node x, which includes all descendants of x in the tree T. Since x_n is the root of the tree, we have $T(A) = T[x_n]$. If $y \in T[x]$, then the node y is a *descendant* of x, and x an *ancestor* of y. Note that every node is an ancestor and a descendant of itself.

To simplify discussion, we shall use $T[x]$ to denote both the subtree itself and the set of nodes in this subtree. This means that, on the one hand, $T[x]$ is a subtree of the elimination tree $T(A)$ and, on the other hand, the node set $T[x]$ defines a subgraph of the graph $G(A)$. No attempt shall be made to distinguish between a set of nodes of $G(A)$ and the subgraph of $G(A)$ that it induces. That is, we shall use $T[x]$ as a node subset and as a subgraph of $G(A)$ interchangeably. Note that the node set $T[x]$ also induces a subgraph in the filled graph $G(F)$.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

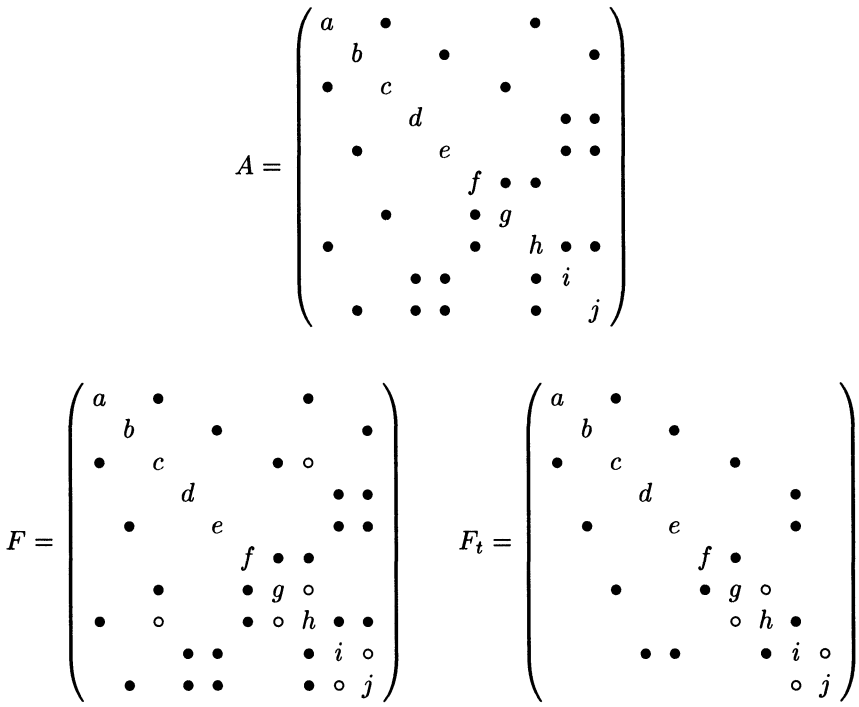


FIG. 2.1. *An example of matrix structures.*

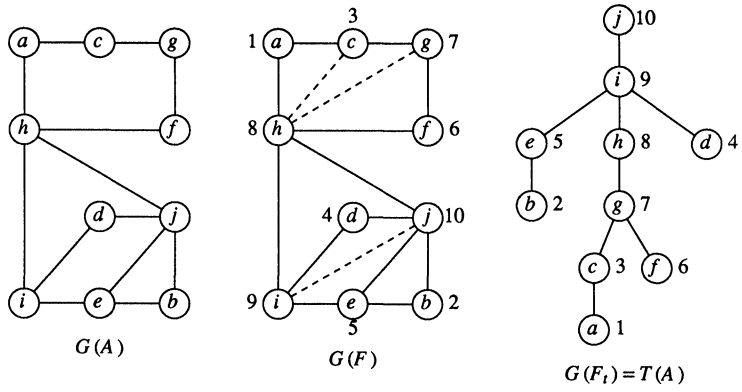


FIG. 2.2. *Graph structures of the example in Fig.* 2.1.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

2.3. Motivation from column dependencies. It is interesting to provide an alternative view to introduce the notion of elimination trees. This motivation comes from the dependency in numerical values among columns of the Cholesky factor. We state the following relation without proof.

PROPOSITION 2.2. *For $i > j$, the numerical values of column L_{*i} depend on column L_{*j} if and only if $\ell_{ij} \neq 0$.* □

The immediate consequence of this observation is that the filled graph $G(F)$ captures the column dependencies among the Cholesky columns. To be precise, if we use a directed edge from node x_j to node x_i to indicate that column i depends on column j, a directed filled graph will give the exact relation. Indeed, for each edge in the filled graph, we simply provide a direction that points from the node with a lower subscript to the higher one. The resulting directed graph is actually the graph of the (unsymmetric) Cholesky factor L^T.

If we are interested only in the column dependency relation, we can simplify this directed graph by a process commonly known in graph theory as *transitive reduction* [1]. That is, if there is a directed path of length greater than one from x_j to x_i, and a directed edge from x_j to x_i, the edge from x_j to x_i is deemed redundant and can be removed. The removal of all such redundant edges from the directed filled graph gives its transitive reduction.

The transitive reduction of the directed filled graph generates precisely the elimination tree structure. We can therefore view the elimination tree as providing the minimal amount of information on column dependencies in the Cholesky factor. To better illustrate this connection, we provide in Fig. 2.3 the directed filled graph of the example in Fig. 2.2. We also include the transitive reduction of this directed graph and the final elimination tree structure.

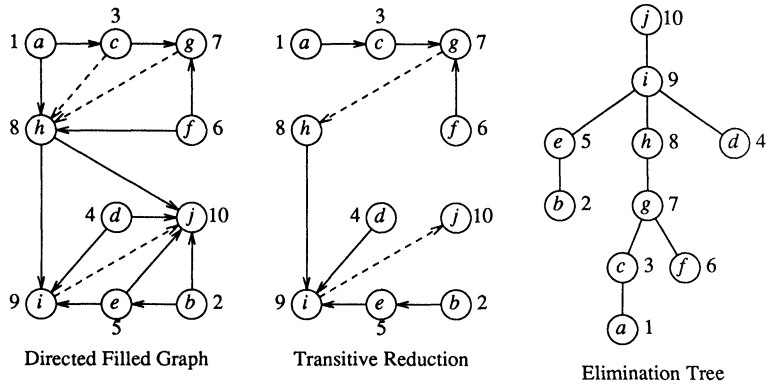


FIG. 2.3. *Directed filled graph and its transitive reduction of the example in Fig.* 2.2.

2.4. Interpretation as a depth-first search tree. As noted earlier, the elimination tree is a spanning tree of the filled graph $G(F)$ of the given matrix A. It can be obtained from the transitive reduction of the directed graph of L^T. In this section, we show that this tree structure can also be obtained from a depth-first search exploration of the undirected filled graph $G(F)$.

Depth-first search is a standard technique of systematically exploring nodes in a graph. It serves as a fundamental tool in devising many efficient graph algorithms (see, for example, [33], [57]). The search starts with an initial node x; and x is marked as *visited.* Then each unvisited node adjacent to x is searched in turn, using depth-first search recursively. The readers are referred to [1] and [58] for details. The edges

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

that lead to new (unmarked) nodes during a depth-first search of a connected graph G form a rooted tree, called a *depth-first search tree.* In other words, if the edge $\{u, v\}$ leads the search from the marked node u to the unmarked node v, then u is the parent node of v in the depth-first search tree. This means the first node visited is the root of this search tree.

An edge of G that connects a node to one of its proper ancestors, except its parent, in a depth-first search tree is called a *back edge.* (Hopcroft and Tarjan [33], [57] use the term *fronds* to refer to back edges). An edge of G that connects two nodes that are not ancestors of each other is called a *cross edge.* The following important property of a depth-first search tree is well known, and its proof can be found in [57].

THEOREM 2.3. [57]. *For a depth-first search tree of an undirected connected graph G, each edge of G is either a tree edge or a back edge of the tree. (That is, there is no cross edge).* □

We provide a different view of the elimination tree in terms of a depth-first search tree in the next theorem. A direct constructive approach is used in the proof, since it gives better insight into the connection.

THEOREM 2.4. *The elimination tree $T(A)$ of a connected graph $G(A)$ is a depth-first search tree of the filled graph $G(F)$ of A.*

Proof. Let $x_1, x_2, \cdots, x_n$ be the node ordering of the filled graph $G(F)$. Consider the depth-first search of $G(F)$ subject to the following tie-breaking rule: when there is a choice of more than one node to explore next, always pick the one with the largest subscript. The rule implies that the search will start with x_n as the initial node.

It remains to show that the resulting depth-first search tree is the same as the elimination tree $T(A)$. Since both are spanning trees of the filled graph $G(F)$, it is sufficient to show that every tree edge of the depth-first search tree is also an edge in the elimination tree. Consider a tree edge from x_p to x_j in the depth-first search tree; that is, during the search, the edge $\{x_p, x_j\}$ in the filled graph leads to the node x_j. We leave it to the reader to verify that the subscript p is greater than j and is indeed the same as $PARENT[j] = \min \{ i > j \mid \ell_{ij} \neq 0 \}$. □

It follows from Theorem 2.4 that properties of depth-first search trees also apply to elimination trees. In particular, by Theorem 2.3, every edge in the filled graph is either a back edge or a tree edge of the elimination tree. In his thesis [48], Peters establishes this observation and refers to the filled graph as a *palm.* (The notion of a palm tree is introduced by Hopcroft and Tarjan [33] to refer to a directed graph with an underlying rooted spanning tree such that every directed edge in the graph connects a node to its ancestor in the tree.)

To illustrate the result of Theorem 2.4, we use the filled graph example of Fig. 2.2. It is easy to verify that

$$j,\ i,\ h,\ g,\ f,\ c,\ a,\ e,\ b,\ d$$

is the sequence of node visits during a depth-first search subject to the tie-breaking strategy in the proof of Theorem 2.4. For example, after the node j has been visited, we can choose any one of b, d, e, h, or i as the next node. Since $i = x_9$ has the largest subscript, it is selected next in the search. After node i, node $h = x_8$ will be selected out of the three possible candidates: d, e, and h. Note that after node f has been visited, there is no unvisited node adjacent to the node f, so that the search backs up to the node g and finds an unvisited neighbor c of g.

In Fig. 2.4, we display the filled graph $G(F)$ and the resulting depth-first search tree. Back edges are also included in the figure and they are represented by curve

directed lines. Without the back edges, the underlying depth-first search tree is identical to the elimination tree of Fig. 2.2.

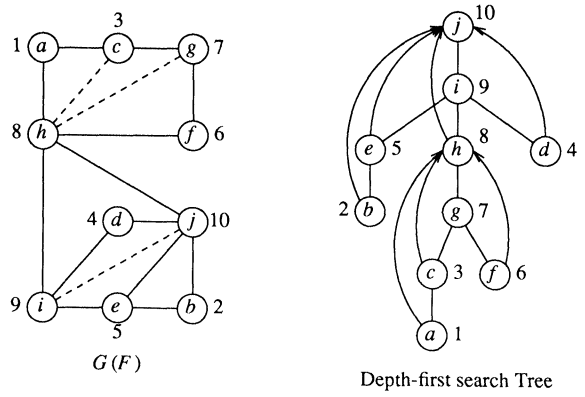


FIG. 2.4. *A depth-first search tree of the filled graph in Fig. 2.2.*

3. Elimination trees and Cholesky factorization.

3.1. Path characterization of filled edges. The elimination tree $T(A)$ is defined in terms of the Cholesky factor L of the matrix A. This tree structure contains a lot of information pertinent to the sparse factorization process. In [55], Schreiber establishes a number of interesting properties of elimination trees that are relevant to the study of sparse Gaussian elimination. The next result is restated here in our terminology without proof. In what follows, unless otherwise stated, we assume that i, j, and k are subscripts and they satisfy $i > j > k$.

THEOREM 3.1. [55]. *If $\ell_{ij} \neq 0$, then the node x_i is an ancestor of x_j in the elimination tree.* □

COROLLARY 3.2. *Let $T[x_i]$ and $T[x_j]$ be two disjoint subtrees of the elimination tree. Then for all $x_s \in T[x_i]$ and $x_t \in T[x_j]$, $\ell_{st} = 0$.*

Proof. Assume for contradiction that there exist nodes $x_s \in T[x_i]$ and $x_t \in T[x_j]$ such that $\ell_{st} \neq 0$. Without loss of generality, let $s < t$. By Theorem 3.1, the node x_t is an ancestor of x_s. This implies that $x_s \in T[x_t] \subseteq T[x_j]$. Therefore, the subtrees $T[x_i]$ and $T[x_j]$ have the node x_s in common and cannot be disjoint. □

Theorem 3.1 provides a necessary condition in terms of the ancestor-descendant relation in the elimination tree for an entry to be nonzero in the filled matrix. Indeed, the result is implicit from the connection between the elimination tree and the depth-first search tree in Theorem 2.4. Every nonzero ℓ_{ij} corresponds to an edge $\{x_i, x_j\}$ in the filled graph so that by Theorem 2.3, it is either a tree edge or a back edge of the elimination tree. Every such edge, therefore, always connects an ancestor-descendant pair in the elimination tree. Corollary 3.2 is simply a restatement of the fact that there is no cross edge in the elimination tree.

We now consider a necessary and sufficient condition for an entry to be nonzero in the filled graph. Rose, Tarjan, and Lueker [53] characterize edges in the filled graph based on the special type of paths in the original graph $G(A)$. We quote the following "path theorem" from them.

THEOREM 3.3. [53]. *Let $i > j$. Then $\ell_{ij} \neq 0$ if and only if there exists a path*

$$x_i, x_{p_1}, \cdots, x_{p_t}, x_j$$

in the graph $G(A)$ such that all subscripts in $\{p_1, \cdots, p_t\}$ are less than j. □

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Note that the path length in this "path theorem" can be one (that is, $t = 0$). The subscript condition implies that any intermediate nodes along this path must be eliminated before x_j (and x_i). We now extend this path condition in terms of subtrees in the elimination tree.

THEOREM 3.4. *Let $i > j$. Then $\ell_{ij} \neq 0$ if and only if there exists a path*

$$x_i, x_{p_1}, \cdots, x_{p_t}, x_j$$

in the graph $G(A)$ such that $\{x_{p_1}, \cdots, x_{p_t}\} \subseteq T[x_j]$.

Proof. If part. Assume that such a path exists. Since x_j is a proper ancestor of every node in $\{x_{p_1}, \cdots, x_{p_t}\}$, by Proposition 2.1, each subscript is less than j. Then by Theorem 3.3, $\ell_{ij} \neq 0$.

Only if part. Assume $\ell_{ij} \neq 0$. By Theorem 3.3, there exists a path

$$x_i = x_{p_0}, x_{p_1}, \cdots, x_{p_t}, x_{p_{t+1}} = x_j$$

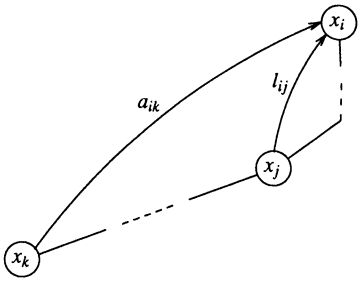
in the graph $G(A)$ such that all subscripts in $\{p_1, \cdots, p_t\}$ are less than j. It remains to show that each node in $\{x_{p_1}, \cdots, x_{p_t}\}$ belongs to $T[x_j]$. There is nothing to prove if $t = 0$. Consider the case when $t > 0$. Assume for contradiction that not every intermediate node belongs to the subtree $T[x_j]$. Let $s > 0$ be the largest subscript such that x_{p_s} does not belong to $T[x_j]$. By the choice of the subscript s, we have $x_{p_{s+1}} \in T[x_j]$.

Since the path is in $G(A)$, this implies that $\{x_{p_s}, x_{p_{s+1}}\}$ is an edge in $G(A)$ and hence also in $G(F)$. But $x_{p_{s+1}}$ cannot be an ancestor of x_{p_s}, for otherwise x_{p_s} would belong to the subtree $T[x_j]$. Therefore, by Theorem 3.1, x_{p_s} must be an ancestor of $x_{p_{s+1}}$. Now that both x_j and x_{p_s} are ancestor nodes of $x_{p_{s+1}}$, and x_j is not an ancestor of x_{p_s}, the node x_{p_s} must be a proper ancestor of x_j. This contradicts the fact that $p_s < j$ and Proposition 2.1. □

3.2. Row structure of the Cholesky factor. Theorems 3.3 and 3.4 characterize edges in the filled graph in terms of paths. In [36], this author extends a property established by Schreiber [55], and the result provides a different necessary and sufficient condition for entries in the Cholesky factor L to be nonzero. We quote this alternative characterization below.

THEOREM 3.5. [36]. *$\ell_{ij} \neq 0$ if and only if the node x_j is an ancestor of some node x_k in the elimination tree, where $a_{ik} \neq 0$.* □

The same result was also given by Tarjan and Yannakakis [59, page 570]. They credited the result to Whitten [60], who presented it at the 1978 SIAM Symposium on Sparse Matrix Computations. Pictorially, the result of Theorem 3.5 can be depicted as follows:



The result of Theorem 3.5 can be used to characterize the row structure of the Cholesky factor. Let us define $T_r[x_i]$ to be the structure of the ith row of the Cholesky

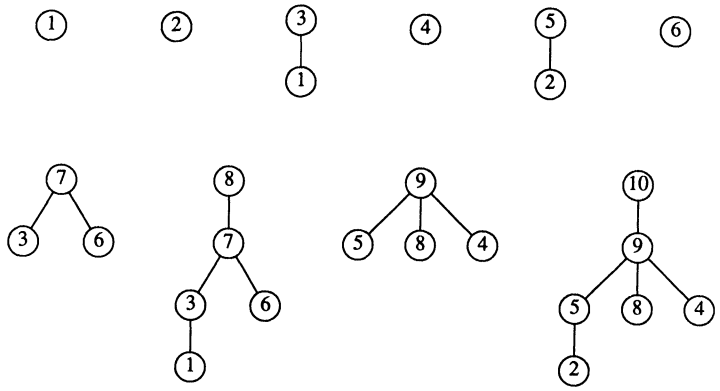


FIG. 3.1. *Row subtrees for the matrix example of Fig.* 2.1.

factor L, that is,

$$T_r[x_i] = \{ x_j \mid \ell_{ij} \neq 0 , j \leq i \} .$$

It follows from Theorem 3.1 that

$$T_r[x_i] \subseteq T[x_i] ,$$

and from Theorem 3.3 that if $a_{ik} \neq 0$, the row structure $T_r[x_i]$ includes all nodes on the path from x_k to x_i in the elimination tree. Schreiber [55] shows that the row structure $T_r[x_i]$ is a pruned subtree rooted at the node x_i in the elimination tree. We shall refer to $T_r[x_i]$ as the ith *row subtree* of L.

Each row subtree $T_r[x_i]$ is completely determined by its set of leaves, since the leaves specify the locations in which the subtree $T[x_i]$ should be pruned. The next corollary characterizes such leaves, and it follows immediately from Theorem 3.5.

COROLLARY 3.6. [36]. *The node x_j is a leaf in the row subtree $T_r[x_i]$ if and only if $a_{ij} \neq 0$, and for every proper descendant x_k of x_j, $a_{ik} = 0$.* □

Note that each leaf x_j in the row subtree $T_r[x_i]$ corresponds to an edge $\{x_i, x_j\}$ in the original graph $G(A)$. In Fig. 3.1, we illustrate the sequence of row subtrees for the example of Fig. 2.1. For convenience, in the figure, each node is labeled by its subscript. Consider the row subtree $T_r[x_{10}]$, which has three leaf nodes $\{x_2, x_4, x_8\}$. Since the node x_7 is the only child of these leaves, the row subtree $T_r[x_{10}]$ can be obtained by pruning the subtree $T[x_7]$ from $T[x_{10}]$.

Therefore, the entire row structure for L is characterized by the elimination tree $T(A)$ and the structure of the original matrix A. The observation that each row structure is a pruned subtree of the elimination tree has a number of interesting applications. It will be used later in this paper to compute the number of nonzeros in L (§3.4), to test if an ordering is perfect elimination (§4.1), to design a compact row storage scheme (§7.2), and to perform symbolic factorization by rows (§8.2).

3.3. Column structure of the Cholesky factor. The previous section characterizes the row structures of the Cholesky factor L in terms of the elimination tree. The result of Theorem 3.1 partially characterizes the column structure of the Cholesky factor. For column j of L, the subscript set of nonzeros in this column is contained in the ancestor set of x_j in the elimination tree. In the next theorem, we provide a complete characterization of the column structures using the elimination tree.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

For a graph G and a node v in G, we use $Adj_G(v)$ to denote the set of nodes adjacent to v in the graph. We further extend this Adj operator to subsets. For a subset S of nodes, we define the adjacent set of S in G to be

$$Adj_G(S) = \{ x \notin S \mid x \in Adj_G(v) \text{ for some } v \in S \} .$$

Note that the adjacent subset $Adj_G(S)$ does not include any nodes in S.

THEOREM 3.7. *The structure of column j of the Cholesky factor is given by*

$$Adj_{G(A)}(T[x_j]) \cup \{x_j\} = \{ x_i \mid \ell_{ij} \neq 0 , i \geq j \} .$$

Proof. Consider column j of the Cholesky factor. Since the diagonal entry $\ell_{jj} \neq 0$, the node x_j belongs to the subset. If $\ell_{ij} \neq 0$ with $i > j$, by Theorem 3.5, this is equivalent to the fact that the node x_j is an ancestor of some node x_k with $a_{ik} \neq 0$. This means $x_i \in Adj_{G(A)}(x_k)$, where x_k belongs to the subtree $T[x_j]$. Or simply, this is equivalent to $x_i \in Adj_{G(A)}(T[x_j])$. □

Consider node c in the example of Fig. 2.2. The subtree $T[c]$ contains nodes a and c, so that

$$Adj_{G(A)}(T[c]) = \{ g, h \} ,$$

which gives precisely the locations of off-diagonal nonzeros in the column associated with the node c in the Cholesky factor of A. We now provide some observations based on the result of Theorem 3.7.

COROLLARY 3.8. *$Adj_{G(A)}(T[x_j]) \cup \{x_j\}$ is a clique in the filled graph $G(F)$.* □

COROLLARY 3.9. *For the subset of nodes $T[x_j]$,*

$$Adj_{G(A)}(T[x_j]) = Adj_{G(F)}(T[x_j]) .$$

Proof. The result follows from Theorem 3.7 and the fact that F is a perfect elimination matrix with no additional fill. □

By Corollary 3.9, we can use the notation $Adj(T[x_j])$ to refer to both $Adj_{G(A)}(T[x_j])$ and $Adj_{G(F)}(T[x_j])$. We do so if the underlying graph is clear from the context. For convenience, we also use $Adj(x_j)$ to refer to $Adj_{G(A)}(x_j)$, which is in general a subset of $Adj_{G(F)}(x_j)$. In other words, the adjacent operator Adj, by default, applies to the graph $G(A)$.

3.4. Nonzero counts of the Cholesky factor. The result of Theorem 3.5 can be used to devise efficient algorithms for counting nonzeros in the Cholesky factor matrix L. Let $\eta(L_{i*})$ and $\eta(L_{*j})$ be the number of nonzeros in the ith row and jth column of the factor L, respectively. These quantities can be computed efficiently using the following algorithm. Here, $marker[*]$ is a working integer vector used in the algorithm to mark nodes that have been considered in the current row structure. A similar description of this algorithm is also given by Zmijewski and Gilbert [62]. In [4], Bank and Smith provide an algorithm that essentially computes the nonzero count of L and the structure of the elimination tree simultaneously.

Algorithm 3.1. Nonzero Count.

```
for j := 1 to n do
    η ( L_* j ) := 1 ;
for i := 1 to n do
begin
```

```
η ( L_i * ) := 1 ;
marker [ x_i ] := i ;
for k < i and a_ik ≠ 0 do
begin { traverse and mark nodes in the row subtree T_r [x_i ] }
   j := k ;
   while marker [ x_j ] ≠ i do
   begin
      η ( L_i * ) := η ( L_i * ) + 1 ;
      η ( L_* j ) := η ( L_* j ) + 1 ;
      marker [ x_j ] := i ;
      j := PARENT[j]
   end
end
end.
```

It is easy to verify that Algorithm 3.1 takes time proportional to the number of nonzeros in the Cholesky factor L and space proportional to the number of nonzeros in the original matrix A. It traverses through nodes in each row subtree $T_r[x_i]$ to obtain the count $\eta(L_{i*}) = | T_r[x_i] |$.

The column nonzero counts $\{\eta(L_{*j})\}$ are also computed in Algorithm 3.1 during the traversal of the row subtrees. These quantities are useful in setting up columnwise storage schemes. This implies that the storage requirement for values of L can be obtained with only the structure of A and the elimination tree. Moreover, the number of arithmetic operations can also be computed using the column nonzero counts. Indeed, if $\eta(L)$ is the number of nonzeros in the factor L and $\mu(L)$ is the number of multiplicative operations to perform the Cholesky factorization, we have [22]

$$\eta(L) = \sum_{j=1}^{n} \eta(L_{*j}) ,$$

$$\mu(L) = \sum_{j=1}^{n} [\eta(L_{*j}) - 1] [\eta(L_{*j}) + 2] / 2 .$$

Therefore the storage and computational cost for the factor L can be determined without the formation of the actual structure of L. This will be a useful utility routine for any sparse matrix package.

The row nonzero counts $\{\eta(L_{i*})\}$ are also useful even in the context of numerical sparse Cholesky factorization by columns. For $1 \leq i \leq n$, $\eta(L_{i*}) - 1$ represents the number of column modifications required to transform this column of A to that of L (see Proposition 9.1 in §9). Some implementations of parallel sparse factorization make use of this information [19], [62].

3.5. Connectivity consideration. The path characterizations of filled edges in Theorems 3.3 and 3.4 provide some relation between fills and connectivity. In this section, we present a more thorough consideration of subtrees in the elimination tree and connected components for subgraphs in $G(A)$ and $G(F)$. As before, we assume that the graph $G(A)$ is connected (or equivalently, the matrix A is irreducible). Recall the subtree notation $T[x]$ introduced at the end of §2. For a given node x, the subtree $T[x]$ identifies a node subset and hence also defines a subgraph of $G(A)$ and of the filled graph $G(F)$.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

THEOREM 3.10. *For each node x_j, the subgraph of $G(A)$ $\{G(F)\}$ consisting of nodes in $T[x_j]$ is connected.*

Proof. We prove by induction on the number of nodes t in $T[x_j]$. The result is obviously true if there is only one node in $T[x_j]$. Assume that the result holds true for all subtrees of size less than t, and $t > 1$. Let $x_{s_1}, \cdots, x_{s_m}$ be the children nodes of x_j. By the inductive assumption, each subgraph consisting of nodes in $T[x_{s_k}]$, for $1 \leq k \leq m$ has fewer than t nodes and is hence connected in $G(A)$. Moreover, for each k, $\{x_j, x_{s_k}\}$ is an edge in the filled graph $G(F)$. By Theorem 3.4, there exists a path from x_{s_k} to x_j through nodes in $T[x_{s_k}]$. This proves the claim that the node subset $T[x_j]$ is a connected subgraph in $G(A)$. Since $G(F)$ is a supergraph of $G(A)$, $T[x_j]$ must also be a connected subgraph in $G(F)$. □

COROLLARY 3.11. *For each node x_j, the set of nodes in $T[x_j]$ forms a connected component in the subgraph of $G(A)$ $\{G(F)\}$ consisting of all nodes except those in $Adj(T[x_j])$.* □

COROLLARY 3.12. *For each node x_j, the set of nodes in $T[x_j]$ forms a connected component in the subgraph of $G(A)$ $\{G(F)\}$ consisting of all nodes except proper ancestors of x_j.*

Proof. Since $Adj(T[x_j])$ is a subset of proper ancestors of the node x_j, the result follows from Corollary 3.11. □

To illustrate these results, consider the example in Fig. 2.2. The subtree $T[g]$ consists of nodes $\{a, c, f, g\}$. This set is a connected subgraph of $G(A)$ (as proved in Theorem 3.10). Furthermore, the adjacent set $Adj(T[g])$ is given by $\{h\}$, whose removal from $G(A)$ gives two connected components, one of which is $\{a, c, f, g\}$. The removal of the set $\{h, i, j\}$ of proper ancestors of g also leaves $T[g]$ as one of the remaining components.

It follows from Corollary 3.11 that if

$$T[x_j] \cup Adj(T[x_j]) \neq X(A),$$

then the subset $Adj(T[x_j])$ separates the nodes of $T[x_j]$ from those of $X(A) - \{T[x_j] \cup Adj(T[x_j])\}$. This observation is used in [45] to devise an effective graph partitioning algorithm by node separators.

4. Elimination trees and chordal graphs.

4.1. Testing chordality. In this section, we shall present some uses of the notion of elimination tree in the study of chordal (or triangulated) graphs. An edge is said to be a *chord* of a cycle if it joins two nonconsecutive nodes on the cycle. An undirected graph is said to be *chordal* if every cycle of length at least four has a chord. It is well known that chordal graphs are exactly those graphs with perfect elimination orderings. Filled graphs are examples of such chordal graphs. A thorough treatment of chordal graphs as a subclass of *perfect graphs* is given by Golumbic [32].

We first consider the use of elimination trees for testing if a given undirected graph is chordal. Since chordal graphs are exactly those with perfect elimination orderings, the chordality of a graph can be established by finding a perfect elimination ordering. Rose, Tarjan, and Lueker [53] and Tarjan and Yannakakis [59] have provided linear algorithms for testing chordality.

One key step in their chordality test algorithms is a linear algorithm that determines if a given ordering is a perfect elimination ordering for a graph $G(A)$. We shall describe this step using the connection of an elimination tree. Let $x_1, x_2, \cdots, x_n$

be the ordering of the graph $G(A)$. For each j, if column j of A does not have any nonzero entry under the diagonal, define $p_j = 0$. Otherwise, let

$$p_j = \min \{ i > j \mid a_{ij} \neq 0 \} .$$

Note that $\{p_j\}$ is defined in the similar way as $PARENT[*]$, except that entries in A are used instead of those in L. Of course, if A does not suffer any fill in its Cholesky factorization, then $PARENT[j] = p_j$.

THEOREM 4.1. *$x_1, x_2, \cdots, x_n$ is a perfect elimination ordering for $G(A)$ if and only if for every row i of A, $a_{i,p_k} \neq 0$ for every nonzero a_{ik} with $k < i$.*

Proof. If part. Assume for contradiction that there exist i and k with $k < i$ such that $a_{ik} \neq 0$ and $a_{i,p_k} = 0$. By the definition of p_k, we have

$$k < p_k \leq i \text{ and } a_{p_k,k} \neq 0 .$$

Therefore, the given ordering cannot be a perfect elimination ordering, since the elimination of the node x_k will create a fill at location ℓ_{i,p_k}.

Only if part. Assume that $x_1, \cdots, x_n$ is a perfect elimination ordering for $G(A)$. Let $T(A)$ be the elimination tree of A. This implies that $PARENT[i] = p_i$ for $i = 1, \cdots, n$. Consider any nonzero a_{ik} with $k < i$. The node x_k belongs to the row subtree $T_r[x_i]$ rooted at x_i. Therefore, the parent node $x_{PARENT[k]} = x_{p_k}$ of x_k also belongs to this row subtree. It follows from Theorem 3.5 that ℓ_{i,p_k} is nonzero. Since the ordering is a perfect elimination ordering for $G(A)$, a_{i,p_k} must also be nonzero. □

Theorem 4.1 contains a simple test to see if an ordering is a perfect elimination ordering for a given graph $G(A)$. In essence, it is comparing to see if the ith row structure of A is the same as the ith row structure of L (which is the row subtree $T_r[x_i]$), for $i = 1, \cdots, n$. The following algorithm is from Tarjan and Yannakakis [59].

Algorithm 4.1. Test for Perfect Elimination Ordering.

```
for i := 1 to n do
begin
   marker [ x_i ] := i ;
   for k < i and a_ik ≠ 0 do
      marker [ x_k ] := i ;
   for k < i and a_ik ≠ 0 do
      if marker[x_{p_k} ] ≠ i then return false ;
   p_i := max { 0, min { s > i | a_si ≠ 0 } }
end;
return true.
```

It is easy to see that Algorithm 4.1 determines if an ordering is perfect elimination for a graph $G(A)$ in time proportional to the number of nodes and number of edges in $G(A)$. The chordality-testing algorithms of Rose, Tarjan, and Lueker [53] and Tarjan and Yannakakis [59] make use of Algorithm 4.1. Both consist of two steps:

Step 1. Find an ordering P for the graph $G(A)$ such that P is a perfect elimination ordering for A if and only if $G(A)$ is chordal.

Step 2. Apply Algorithm 4.1 to see if the ordering P is a perfect elimination ordering for $G(PAP^T)$.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

It is clear from definitions that a *minimal fill ordering*[1] for A is a perfect elimination ordering if and only if $G(A)$ is chordal. Therefore, any minimal fill ordering will serve the purpose for Step 1. The *lexicographic ordering*, a special form of breadth-first search, by Rose, Tarjan, and Lueker [53] can be used to determine a minimal fill ordering in linear time. Tarjan and Yannakakis [59] provide a simpler linear algorithm based on what they call *maximum cardinality search*, to determine an ordering satisfying the requirement in Step 1. Combining Algorithm 4.1 with either the lexicographic ordering or the maximum cardinality search, we have therefore an overall linear algorithm for testing the chordality of undirected graphs. It is interesting to note the implicit role played by the elimination tree in these chordality-testing algorithms.

4.2. Intersection graph representations of chordal graphs. It is known that a graph is chordal if and only if it is the *intersection graph* of a family of subtrees of a tree [17], [32]. In this section, we show how to construct such a tree representation explicitly using the elimination tree of a chordal graph. The approach of using elimination trees in arriving at this tree representation is new.

For completeness, we first define intersection graphs. Let Γ be a family of nonempty sets. The *intersection graph* of Γ is obtained by representing each set in Γ by a node and connecting two nodes by an edge if and only if their corresponding sets intersect (see, for example, Golumbic [32]).

Let A be a symmetric matrix with graph structure $G(A)$ and filled graph $G(F)$. We assume that $x_1, x_2, \cdots, x_n$ is the node elimination sequence. Let $T(A)$ be the corresponding elimination tree, with $T_r[x_1], \cdots, T_r[x_n]$ as the sequence of row subtrees.

LEMMA 4.2. $\ell_{ij} \neq 0$ *if and only if* $T_r[x_i] \cap T_r[x_j] \neq \emptyset$.

Proof. For definiteness, assume $i > j$. It follows from definition that $\ell_{ij} \neq 0$ if and only if $x_j \in T_r[x_i]$. Therefore, if $\ell_{ij} \neq 0$, then $x_j \in T_r[x_i] \cap T_r[x_j]$ and hence $T_r[x_i] \cap T_r[x_j] \neq \emptyset$.

On the other hand, if the intersection of the two row subtrees is nonempty, say $x_s \in T_r[x_i] \cap T_r[x_j]$, this implies that the node x_j lies on the path from x_s to x_i in the elimination tree. Hence $x_j \in T_r[x_i]$ and that implies that $\ell_{ij} \neq 0$. □

THEOREM 4.3. *The chordal graph $G(F)$ is the intersection graph of the row subtrees in the elimination tree $T(A)$.*

Proof. The result follows directly from Lemma 4.2. □

The result in Theorem 4.3 gives a constructive approach to determine the intersection graph representation of any chordal graph. Indeed, for a given chordal graph G, we can first find a perfect elimination sequence $x_1, \cdots, x_n$ for G. Obtain the elimination tree of this node sequence and the associated row subtrees $T_r[x_1], \cdots, T_r[x_n]$. Then the chordal graph G is given by the intersection graph of these row subtrees in the elimination tree.

4.3. Separators for chordal graphs. A *separator* of a connected graph is a subset of nodes whose removal renders the remaining subgraph disconnected. In [30], Gilbert, Rose, and Edenbrandt have devised an efficient algorithm to determine $O(\sqrt{m})$-separators for chordal graphs, where m is the number of edges in the graph. Their algorithm uses implicitly the structure of an elimination tree. In this section, we explore this connection. Let A be a sparse matrix with $G(A)$ as a chordal graph and assume that it is ordered with no fill. Let $T(A)$ be its corresponding elimination tree.

[1] A minimal fill ordering P on the matrix A is one such that no ordering of A will generate a filled graph that is a proper subgraph of the filled graph of PAP^T [52].

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

The separator algorithm by Gilbert, Rose, and Edenbrandt [30] finds a set of size at most $O(\sqrt{m})$, whose removal divides the graph into connected components, each with no more than $n/2$ nodes. Their approach can be interpreted as first obtaining from the elimination tree $T(A)$ the smallest subscript j such that $T[x_j]$ has more than $n/2$ nodes. Then, the desired $O(\sqrt{m})$-separator is given by the set $S = Adj(T[x_j]) \cup \{x_j\}$.

It follows from Corollary 3.8 that this set S forms a clique in the chordal graph. Being a clique, the set must have size no greater than $O(\sqrt{m})$. Furthermore, by the construction of S, the removal of S leaves no component with more than $n/2$ nodes. Our discussion here presents a different view of the separator result in the paper [30] and interprets the algorithm in terms of the elimination tree.

5. Determination of elimination trees.

5.1. Basis of the algorithm. For an n-by-n symmetric matrix A, we consider the problem of determining the structure of its elimination tree $T(A)$, that is, computing the parent vector $PARENT[i]$, $i = 1, \cdots, n$, of $T(A)$. We can obtain the parent vector using the following code.

```
for i := 1 to n do
begin
   PARENT[i] := 0 ;
    for k < i and ℓ_ik ≠ 0 do
        if PARENT[k] = 0 then PARENT[k] := i ;
end.
```

Note that the inner **for**-loop uses the row structures of the Cholesky factor L. For this purpose, we can generate the location of each row nonzero of L as needed, without storing the entire structure of L. The row structure characterization of the factor matrix in §3.2 is applicable here. Indeed, for each row i, we can generate the structure of row i of L using the structure of the original matrix A and the current values of $PARENT[k]$, $k = 1, \cdots, i-1$. That will result in an algorithm for computing the parent vector in time proportional to the number of nonzeros in L and in space proportional to the number of nonzeros in A.

5.2. The algorithm using set union operations. In [36], Liu further extends the row approach of the last section to obtain more efficient ways of finding the elimination tree structure. The key observation is that the problem of computing the $PARENT[*]$ vector of $T(A)$ from the graph $G(A)$ can be expressed in terms of basic set operations for the set union problem. Following Tarjan [58, Chap. 2], we consider the three set operations:

makeset(x): create a new singleton set with element x;
find(x): return the representative of the set containing x;
link(x, y): form the union of the two sets containing x and y, and return the new representative of the union set.

Implicitly assumed here is that each set is represented by a rooted tree, where each tree node corresponds to a member of the set, and the root is the representative.

We now describe the determination of the elimination tree from the graph $G(A)$. The following algorithm finds the $PARENT[*]$ vector for the elimination tree $T(A)$ using the above three basic set operations. A temporary vector *realroot*$[*]$ is used to store the actual root of the subtree under consideration in the partially formed elimination tree.

Algorithm 5.1. Elimination Tree.

```
for i := 1 to n do
begin
   makeset( i ) ;
   realroot[i] := i ;
   PARENT[i] := 0 ;
   vroot := i ;
   for k < i and a_ik ≠ 0 do
   begin
      u := find(k);
      t := realroot[u];
      if PARENT[t] = 0 and t ≠ i then
      begin
         PARENT[t] := i ;
         vroot := link(vroot, u) ;
         realroot[vroot] := i
      end
   end
end.
```

Within this framework, the various enhancement techniques as described by Tarjan in [58] can be applied. In particular, the notion of *path compression* to change the structure of the tree during a "*find*" by moving nodes closer to the root can be used to reduce the amount of time for later "*find*"s. In that case, two structures are maintained throughout the course of the algorithm, one for the actual elimination tree and the other for the compressed tree structure. Moreover, the use of *union by ranks* in the "*link*" operation keeps the resulting tree shallow by choosing a more appropriate representative for the union set. It has the desirable effect of balancing the tree.

In [36], Liu provides a detailed description of the algorithm to determine the elimination tree structure using path compression. He also notes that path compression with balancing will produce a theoretically more efficient scheme. An explicit description of this scheme then appears in the work of Zmijewski and Gilbert [62]. We refer the reader to these two papers.

A direct use of the analysis by Tarjan [58] provides the following complexity bounds. Let m be the number of nonzeros in the matrix A.

THEOREM 5.1. *Algorithm 5.1, using only path compression, determines the elimination tree in* $O(m \log_2 n)$ *time.* □

THEOREM 5.2. *Algorithm 5.1, using path compression with balancing, determines the elimination tree in* $O(m\alpha(m,n))$ *time, where* $\alpha(*,*)$ *is a functional inverse of Ackerman's function.* □

In [16], Fischer defines a class of trees in order to establish a lower bound for the set union algorithm without balancing. It is interesting to note that we can use the same example to construct sparse matrix structures so that Algorithm 5.1 with only path compression will require time proportional to $m \log_2 n$. In other words, the time bound in Theorem 5.1 is the best possible.

It is appropriate here to mention two parallel algorithms to determine the elimination tree structure. Zmijewski and Gilbert [62] provide a distributed version of Algorithm 5.1 using both path compression and balancing. Gilbert and Hafsteinsson

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

[29] use the divide-and-conquer approach to devise an efficient parallel algorithm to determine the tree structure.

5.3. Experimental results on balancing. The time bound in Theorem 5.2 involves the inverse Ackerman function α, which is an extremely slow growing function. For all practical purposes, it can be regarded as a constant value of 5. (For more details on this function, the readers are referred to [58].) From the theoretical complexity bound, Algorithm 5.1 using path compression with balancing is superior to the one without balancing.

Both schemes (with and without balancing) were implemented to compute the structure of elimination trees of large sparse matrices. We applied them to the model square grid problem (with 9-point difference operator), where the nodes were ordered by the minimum degree ordering algorithm. The time in CPU seconds on a SUN 3/50 is tabulated in Table 5.1.

TABLE 5.1
Time to determine elimination trees.

Grid	Algorithm 5.1 using path compression	
	without balancing	with balancing
30×30	0.10	0.18
50×50	0.26	0.50
80×80	0.82	1.30
100×100	1.08	2.08
180×180	3.82	6.66

We note from Table 5.1 that the scheme without balancing runs almost twice as fast as the one with balancing, even when the grid size is 180 ($n = 32,400$). Runs on practical sparse matrix examples from the Harwell–Boeing collection give similar results. Therefore, in practice, we recommend the simpler code of using Algorithm 5.1 without balancing.

6. Elimination trees and matrix reordering.

6.1. Topological orderings. In this section, we consider *equivalent* reorderings based on the structure of the elimination tree. Let A be a given symmetric matrix. Following [43], we call two orderings P and Q equivalent if the structures of the filled graphs of PAP^T and QAQ^T are the same (that is, the filled graphs are isomorphic). For convenience, we shall call P an equivalent reordering of the matrix A if the filled graph of A has the same structure as that of PAP^T. It is known that equivalent orderings require the same amount of arithmetic for the sparse Cholesky decomposition of their permuted matrices (see, for example, [11], [52]). Therefore, in terms of both storage and computational costs, equivalent reorderings are as good as the original ordering. However, we may use them to take advantage of other aspects of elimination, some of which will be illustrated in later sections.

The structure of an elimination tree provides some flexibility and freedom in reordering the nodes. We first consider the class of topological orderings. As defined in §2, a topological ordering on a rooted tree is one that numbers the children nodes before their parent node. Given an initial node ordering on the matrix A and its corresponding elimination tree $T(A)$, let P be a permutation matrix for A that corresponds to a topological ordering of the nodes in $T(A)$.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

THEOREM 6.1. *As unlabeled graphs, the filled graph of* $G(PAP^T)$ *and the filled graph of* $G(A)$ *are isomorphic.*

Proof. Let F be the filled matrix of A. Put $\tilde{A} = PAP^T$ and let $\tilde{F}$ be the corresponding filled matrix of $\tilde{A}$. We want to show that $G(F)$ and $G(\tilde{F})$ are structurally identical.

Let $x_1, x_2, \cdots, x_n$ be the node elimination sequence for the matrix A and let $\tilde{x}_1, \tilde{x}_2, \cdots, \tilde{x}_n$ be that for the matrix $\tilde{A}$. Consider two nodes x_i and x_j in A. Let $x_i = \tilde{x}_{\tilde{r}}$ and $x_j = \tilde{x}_{\tilde{s}}$ in the sequence for $\tilde{A}$. It is sufficient to show that $\{x_i, x_j\}$ is an edge in the filled graph $G(F)$ if and only if $\{\tilde{x}_{\tilde{r}}, \tilde{x}_{\tilde{s}}\}$ is an edge in $G(\tilde{F})$.

Assume that $\{x_i, x_j\}$ is an edge in $G(F)$ and $i > j$. By Theorem 3.1, x_i is a proper ancestor of x_j in the elimination tree $T(A)$. Since P is a topological ordering, the node $\tilde{x}_{\tilde{r}}$ is labeled after $\tilde{x}_{\tilde{s}}$ so that $\tilde{r} > \tilde{s}$. Furthermore, by Theorem 3.4, there exists a path in the graph $G(A)$

$$\tilde{x}_{\tilde{r}} = x_i, x_{p_1}, \cdots, x_{p_t}, x_j = \tilde{x}_{\tilde{s}}$$

such that $\{x_{p_1}, \cdots, x_{p_t}\} \subseteq T[x_j]$. By the property of the topological ordering of P, these nodes $x_{p_1}, \cdots, x_{p_t}$ are labeled before $\tilde{x}_{\tilde{s}}$ in the matrix $\tilde{A}$. Therefore, by Theorem 3.3, $\{\tilde{x}_{\tilde{r}}, \tilde{x}_{\tilde{s}}\}$ is also an edge in the filled graph $G(\tilde{F})$.

Conversely, let $\{\tilde{x}_{\tilde{r}}, \tilde{x}_{\tilde{s}}\}$ be an edge in $G(\tilde{F})$. For definiteness, let $\tilde{r} > \tilde{s}$. Note first that x_i does not belong to the subtree $T[x_j]$, for otherwise, it contradicts the topological ordering property of P. By Theorem 3.3, there exists a path in $G(A)$

$$\tilde{x}_{\tilde{r}}, \tilde{x}_{\tilde{p}_1}, \cdots, \tilde{x}_{\tilde{p}_t}, \tilde{x}_{\tilde{s}}$$

such that all subscripts $\tilde{p}_1, \cdots, \tilde{p}_t$ are less than $\tilde{s}$. By the property of the topological ordering of P, the nodes in $\{\tilde{x}_{\tilde{p}_1}, \cdots, \tilde{x}_{\tilde{p}_t}\}$ cannot be ancestors of x_j. From the above path, we know that all the nodes on the path belong to the connected component containing the node x_j in the subgraph of $G(A)$ excluding the set of proper ancestors of x_j in the tree. By Corollary 3.12, these nodes all belong to the subtree $T[x_j]$. In other words, we have a path in $G(A)$ from $\tilde{x}_{\tilde{s}} = x_j$ to $\tilde{x}_{\tilde{r}} = x_i$ through nodes in the subtree $T[x_j]$ and x_i is outside of $T[x_j]$. Again by Corollary 3.12, this means that x_i is a proper ancestor of x_j so that $i > j$. Therefore, using Theorem 3.4, $\{x_i, x_j\}$ is also an edge in the filled graph $G(F)$. □

COROLLARY 6.2. *As unlabeled trees, the elimination tree* $T(PAP^T)$ *and the elimination tree* $T(A)$ *are isomorphic.* □

The result of Theorem 6.1 implies that every topological ordering of the elimination tree is an equivalent reordering of the given sparse matrix. The amount of fills and the number of arithmetic operations for the factorization will be preserved by every topological ordering of the elimination tree. Indeed, by Corollary 6.2, even the structure of the elimination tree is preserved.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

For a given elimination tree, there are many possible topological orderings. One important example is the class of *postorderings* [1]. In a postordering, the nodes within every subtree of the elimination tree will be numbered consecutively. The root of a subtree will always be labeled last among nodes in the subtree. For the elimination tree of Fig. 2.2, the ordering P

$$d,\ f,\ a,\ c,\ g,\ h,\ b,\ e,\ i,\ j$$

is a postordering on the tree $T(A)$. In Fig. 6.1, we display the structure of the filled matrix $\tilde{F}$ of PAP^T, the graph $G(\tilde{F})$ and the tree $T(PAP^T)$.

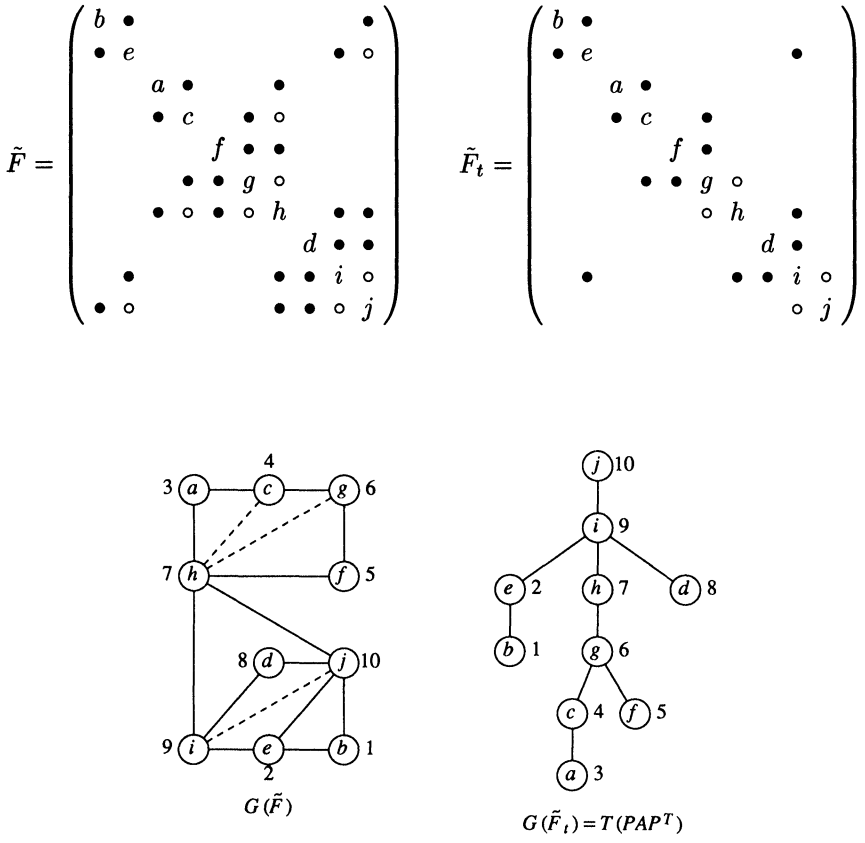


FIG. 6.1. *A postordering of the elimination tree in Fig. 2.2.*

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Topological orderings on an elimination tree are notions similar to *consistent* orderings introduced by Peters [48]. He also provides a proof of a result equivalent to Theorem 6.1. It leads him to observe that if $A = LL^T$ and P is a topological ordering of $T(A)$, then PLP^T is lower triangular and is the Cholesky factor of PAP^T. Furthermore, his concept of *preserving palm* can also be expressed in terms of topological orderings on the elimination tree.

6.2. Root selection of elimination trees by reorderings. In the previous section, for a given sparse matrix A, we have shown that every topological ordering of its elimination tree $T(A)$ is an equivalent reordering for A. Furthermore, the structure of the elimination tree is preserved by such topological orderings. However, in general, equivalent reorderings for A may not retain the elimination tree structure.

We now show that for any node x in $G(A)$, there is an (essentially) equivalent reordering such that the resulting elimination tree is rooted at x. Let F be the filled matrix of A. We need the following result from Rose [52].

THEOREM 6.3. [52]. *For any node x in $G(F)$, there is a perfect elimination ordering on $G(F)$ such that the node x is numbered last.* □

COROLLARY 6.4. *For any node x in $G(A)$, there is an ordering P on $G(A)$ such that the node x is numbered last, and such that the filled graph of $G(PAP^T)$ is a subgraph of the filled graph of $G(A)$.*

Proof. Let $G(F)$ be the filled graph of $G(A)$. By Theorem 6.3, there is a perfect elimination ordering P on $G(F)$ such that the node x is numbered last. In other words, the factorization of the permuted filled matrix PFP^T does not create additional fill. This implies that fills created in the factorization of PAP^T will be accounted for by $G(PFP^T)$. Therefore the filled graph of $G(PAP^T)$ is a subgraph of $G(PFP^T) = G(F)$. □

In Corollary 6.4, the ordering P is at least as good as the initial ordering for A in terms of fill. In the case when the matrix A has been ordered initially by a minimal fill ordering, then the filled graph of $G(PAP^T)$ will be the same as the filled graph of $G(A)$. Otherwise, the ordering P may be a better ordering so that the filled graph of $G(PAP^T)$ is a proper subgraph of the filled graph of $G(A)$. The following theorem is a direct consequence of Corollary 6.4.

THEOREM 6.5. *For any node x in $G(A)$, there is an ordering P on $G(A)$ such that the node x is the root of the elimination tree $T(PAP^T)$, and that the filled graph of $G(PAP^T)$ is a subgraph of the filled graph of $G(A)$.* □

For a better understanding of the result, it is instructive to consider the graph example of Fig. 2.2 (or Fig. 6.1). Say, we want a reordering on $G(A)$ such that node b is the root of the elimination tree. One such reordering is

$$a,\ c,\ f,\ g,\ h,\ d,\ i,\ e,\ j,\ b\,.$$

For this reordering, we display the new labeling and its corresponding elimination tree in Fig. 6.2. Note that the structure of the resulting elimination tree is quite different from the original one in Fig. 2.2. For this example, the filled graph structure is preserved, since the original ordering is a minimal fill ordering (in fact, a minimum fill ordering).

6.3. Tree restructuring by elimination tree rotations. Reordering so that a given node x becomes the root of the resulting elimination tree is a special case of a more general technique, called *elimination tree rotation*, introduced by the author [43]. The essence of elimination tree rotation can be described as follows. For any given

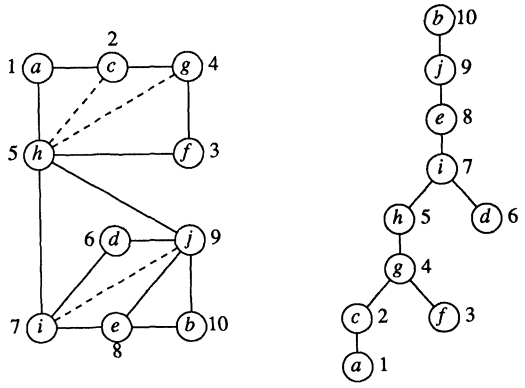


FIG. 6.2. *An equivalent reordering of the graph in Fig.* 2.2.

node x in the elimination tree, the ordering obtained from an elimination tree rotation (actually a composite of basic rotations) at x will maintain the relative ordering of the nodes that are not ancestors of x. But the ancestors nodes of x will be reordered such that the nodes in $Adj(T[x]) \cup \{x\}$ are numbered last.

The overall effect of such a reordering will retain the structure of subtrees outside the ancestor set of x. Moreover, the nodes in $Adj(T[x]) \cup \{x\}$ will be "promoted" up, one of which will form the root of the resulting elimination tree. Since this set is a clique in the filled graph (by Corollary 3.8), they can be renumbered among themselves in any order so long as they are labeled last. Picking x to be the last node will give a tree rooted at x.

The matrix reordering example of Fig. 6.2 is obtained by an elimination tree rotation of the tree in Fig. 6.1 at the node b. Therefore, the nodes in the clique of $G(F)$

$$Adj(T[b]) \cup \{b\} = \{ b, e, j \}$$

are labeled last in the reordering. Of these three nodes, b is selected as the last so that the resulting elimination tree will be rooted at b. The relative ordering of the remaining seven nodes is maintained in the new reordering.

Orderings from elimination tree rotations form an important class of equivalent reorderings in restructuring the tree. The criterion on the tree structure for the best equivalent reordering depends on the application. For example, having a balanced (or unbalanced) tree structure may be a desirable criterion. Another one would be to reduce the height of the elimination tree. Some applications will be discussed in later sections. For more details on elimination tree rotations, the reader is referred to [43] and [46].

7. Elimination trees in sparse storage schemes.

7.1. Relative row-index scheme for Cholesky factors. The most commonly used storage scheme for sparse Cholesky factors is the one proposed by Sherman [56]. His compressed column storage scheme stores the nonzeros in the Cholesky factor column by column. The subscript information is provided in an auxiliary vector in a compressed form by taking advantage of columns whose initial row subscripts are final subsequences of those in the previous column. The scheme maintains *absolute* row index information for the column structures. The relevance of this storage scheme in sparse factorization is also addressed in [15] and [54].

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In [55], Schreiber suggests the use of *relative* row indices to store the structural information for columns of the Cholesky factor. The possible advantages, as pointed out by Schreiber, are: improved efficiency in performing column updates, better utilization of vectorized machines, reduced overhead storage requirement (by packing more relative row indices into a word). One important use of relative indices is in some implementations of the multifrontal method [2].

We now provide the basis for his scheme using the terminology established in this paper. As before, let A be a given sparse symmetric positive-definite matrix initially ordered by some fill-reducing ordering. Let $T(A)$ be the corresponding elimination tree.

THEOREM 7.1. *Let x_p be the parent of the node x_j in the elimination tree. Then*

$$Adj(T[x_j]) \subseteq Adj(T[x_p]) \cup \{x_p\} .$$

Proof. The result follows from the fact that $T[x_j] \subseteq T[x_p]$. □

By Theorem 3.7, the set $Adj(T[x_j]) \cup \{x_j\}$ provides the structure of column j of the Cholesky factor. Theorem 7.1 implies that the structure of column j without the diagonal element is contained in the column structure of p. (Note that in matrix terms, p is the row subscript of the first off-diagonal nonzero in column j of the factor L.) The relative row-index scheme of Schreiber makes use of this observation. For an off-diagonal nonzero ℓ_{ij} in column j, it follows from Theorem 7.1 that ℓ_{ip} is also nonzero. Instead of storing the absolute row index i for the nonzero ℓ_{ij}, Schreiber suggests storing the relative location of the corresponding entry ℓ_{ip} in the structure of column p.

Let us use the matrix example in Fig. 2.1 to illustrate Schreiber's scheme. Using the given ordering, we consider column $j = 2$ or node b. The parent of b in the elimination tree is the node e so that $p = 5$. The structure of column 2 is given by

$$2, 5, 10,$$

while that of column 5 is

$$5, 9, 10.$$

These absolute row subscripts will be stored in a compressed form by Sherman's storage scheme. However, in Schreiber's relative row-index scheme, the structure of column 2 is maintained implicitly by the following sequence of relative indices:

$$-, 1, 3,$$

since the absolute indices of 5 and 10 appear in locations 1 and 3 of column $p = 5$. It is important to realize that the structure of the parent column p will also be stored in the relative row-index form.

Schreiber [55] provides a complete set of numerical Cholesky factorization and forward/backward substitution algorithms using this relative row-index storage scheme. He employs the fact that all column modifications of column j in the Cholesky factorization are from columns $k < j$, where $x_k \in T_r[x_j]$, the row subtree of x_j (see Proposition 9.1 in §9). Moreover, the actual numerical values from such a column k can be passed onto column j through the columns associated with nodes on the path from x_k to x_j in the elimination tree. Therefore, the relative row indices are sufficient to accumulate contributions to column j. The reader is referred to [55] for details.

7.2. Compact row scheme for Cholesky factors. The compressed column storage scheme by Sherman is especially suited for the column-Cholesky factorization algorithm. Moreover, the compressed subscript approach is demonstrated to be very

effective in reducing the amount of overhead storage for the structure of the Cholesky factor. The amount of subscript overhead used is problem dependent.

In [36], Liu proposes an alternative row storage scheme, and the integer overhead for subscripts is shown to be no more than the number of nonzeros in the original matrix A. The scheme is based on the result on the row structure of the Cholesky factor established in §3.2. Since the structure of row i of the factor is given by the ith row subtree (see, for example, Fig. 3.1), it is sufficient to store the *leaves* of the row subtree. But each such leaf corresponds to one nonzero entry in the original matrix (by Theorem 3.5). This implies that for row i, its row subtree structure can be represented by no more than the number of nonzeros in row i of the matrix A.

In this row storage scheme, whenever the structure of row i is needed, its entire set of subscripts will be generated from the row subtree. This is accomplished by a *postorder* tree traversal of the subtree. But in order to obtain the set of (column) subscripts for the row in ascending sequence, we perform an equivalent reordering of the matrix using a postordering (as described in §6.1). Subsequent postorder traversal of any subtree will always produce subscripts in ascending order. It is instructive to note that this observation is not applicable to the row subtrees in Fig. 3.1, since the ordering used is not a postordering of the elimination tree (see Fig. 2.2). However, it works for the reordering used in Fig. 6.1. For more details of the row storage scheme, the reader is referred to the paper [36].

Bank and Smith [4] propose a row scheme, which is quite similar to the one described in [36]. The techniques of postorder reordering and elimination tree computation are also applicable to their scheme.

7.3. Data structures for sparse QR and LU factors. In [27], George and Ng describe a novel implementation of sparse Gaussian elimination with partial pivoting. For a given sparse matrix M (not necessarily symmetric), their scheme determines from the structure of M a *static* data structure that will accommodate nonzeros in the factor matrices for all possible partial pivoting sequences. Their static data structure consists of two parts: one for the lower triangular factor and the other for the upper.

Define the symmetric matrix $A = M^T M$. The static structure for the upper triangular factor of M is given by the Cholesky factor of the symmetric matrix A. Indeed, for any permutation P, if the row-permuted matrix PM is decomposed by Gaussian elimination into LU, George and Ng show that the structure of U is contained in that of the Cholesky factor of $A = M^T M$.

As for the lower triangular part, it turns out that its structure can be determined in terms of the elimination tree $T(A)$ of the matrix product A. It is shown in [26] that each row structure of the lower triangular factor is contained in the structure given by a chain of tree edges in the elimination tree $T(A)$, that is, a sequence of nodes with subscripts

$$i_1,\ i_2 = PARENT[i_1]\,,\cdots,\ i_t = PARENT[i_{t-1}]\,.$$

This not only saves overhead storage in implementing the static data structure, but also reduces the factorization time. It is beyond the scope of this paper to consider the details. The reader is referred to [26]. It should be pointed out that the same static data structure is also useful in the sparse orthogonal decomposition of the matrix M.

8. Elimination trees in symbolic factorization.

8.1. Symbolic factorization of A by columns. Efficient algorithms have been devised to perform the symbolic factorization of a given large sparse symmetric matrix

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

A. In the literature, symbolic factorization refers to the determination of the column structures of the Cholesky factor L of A. Rose, Tarjan, and Lueker [53] provide an algorithm (called "FILL_IN") for this purpose. In [21], George and Liu describe a similar symbolic factorization scheme, the output of which is tailored for the compressed column storage scheme of Sherman.

In this section, we relate the basic algorithm for symbolic factorization in [21] and [53] to the structure of an elimination tree. Indeed, both approaches are based on the following result, which can be regarded as an extension of Theorem 7.1.

THEOREM 8.1. *For any node x_j in the elimination tree, $Adj(T[x_j])$ is given by*

$$(Adj(x_j) - \{x_1, \cdots, x_{j-1}\}) \cup \{ Adj(T[x_s]) - \{x_j\} \mid x_s \text{ is a child of } x_j \} .$$

Proof. By definition of $T[x_j]$, we have

$$Adj(x_j) - \{ x_1, \cdots, x_{j-1} \} \subseteq Adj(T[x_j]) .$$

By Theorem 7.1, for each child node x_s of x_j,

$$Adj(T[x_s]) - \{x_j\} \subseteq Adj(T[x_j]) .$$

On combining the two, we therefore have shown that the given set is contained in $Adj(T[x_j])$.

On the other hand, consider any $x_i \in Adj(T[x_j])$. From Proposition 2.1, note that x_i cannot be one of $\{x_1, \cdots, x_j\}$. By Theorem 3.7, $\{x_i, x_j\}$ is an edge in the filled graph of A. By Theorem 3.5, there exists a node x_k such that the node x_j is an ancestor of x_k and $\{x_k, x_i\}$ is an edge in $G(A)$. If $x_k = x_j$, then clearly $x_i \in Adj(x_j)$. Otherwise, let x_s be the child node of x_j, which is also an ancestor of x_k. Then by Theorem 3.5, $\{x_i, x_s\}$ is also an edge in the filled graph. Therefore, by Theorem 3.7, $x_i \in Adj(T[x_s])$. In other words, in both cases, x_i must also belong to the given set. This completes the proof. □

Theorem 8.1 provides the basis for the formulation of an efficient symbolic factorization algorithm to compute the column structure $\{Adj(T[x_j])\}$. Such formulation requires the structure of the elimination tree $T(A)$. However, since the parent of the node x_j $(j < n)$ in the elimination tree can be obtained readily from $Adj(T[x_j])$, a predetermination of the elimination tree structure is not necessary. We can therefore formulate the following symbolic factorization algorithm.

Algorithm 8.1. Symbolic Factorization by Columns.

```
for j := 1 to n do
begin
   Adj(T[x_j]) := Adj(x_j) - { x_1, ..., x_{j-1} } ;
   for x_s in the children list of x_j do
      Adj(T[x_j]) := Adj(T[x_j]) ∪ Adj(T[x_s]) - {x_j} ;
   if Adj(T[x_j]) ≠ ∅ then
   begin
      p := min { i | x_i ∈ Adj(T[x_j]) } ;
      add x_j to the children list of x_p
   end
end.
```

Algorithm 8.1 uses implicitly the fact that $x_1, \cdots, x_n$ is a topological ordering of the elimination tree $T(A)$. Indeed, when the structure of column j is determined,

we assume that the column structures of its children nodes have all been computed. This is always the case if we compute the column structures in increasing order of the subscript j (from Proposition 2.1).

In terms of the actual implementation of the children lists, we need to use only one extra integer n-vector $CLIST[*]$. It can be arranged so that at the beginning of step j, the list of children nodes of the node x_j in the elimination tree is given by the sequence

$$CLIST[j],\ CLIST[CLIST[j]], \cdots .$$

The space for this list can be reused for children lists of subsequent nodes. In other words, at step j, $CLIST$ stores the complete children list for the node x_j and partial children lists for nodes x_i, with $i > j$. This makes it possible to represent the n children lists during the course of the algorithm using only an n-vector. The symbolic factorization routine in SPARSPAK [22] uses this observation.

8.2. Symbolic factorization of A by rows. In an unpublished manuscript [60], Whitten describes a symbolic factorization algorithm that will determine the structure of the Cholesky factor L by rows. The algorithm can be viewed as using the row structure characterization of L in terms of row subtrees of the elimination tree (Theorem 3.5). For each row i of L, we traverse the row subtree $T_r[x_i]$ as in Algorithm 3.1 for computing the number of nonzeros in L. The nodes visited are collected to form the ith row structure of L. A detailed description of this approach is given by Tarjan and Yannakakis [59].

An interesting feature of this algorithm is its potential for parallelism. Assume that the parent vector of the elimination tree has been precomputed. Then the structures of all n rows can be computed totally independent of each other. Zmijewski and Gilbert [62] use this observation to design a parallel symbolic factorization scheme for message-passing multiprocessors.

8.3. Symbolic factorization of $A = M^TM$ using M. In the sparse orthogonal factorization of a matrix M [20], [38] or the static storage approach of sparse partial pivoting [27] of a square matrix M, it is necessary to compute the structure of the Cholesky factor of the symmetric matrix M^TM. One obvious approach is to construct explicitly the structure of $A = M^TM$ and apply the symmetric symbolic factorization algorithm to A.

In [28], George and Ng provide an efficient algorithm to perform the symbolic factorization of M^TM using the structure of the matrix M. This removes the redundant step of determining the structure of M^TM. Although their algorithm description does not involve elimination trees, their scheme can also be formulated nicely in terms of such trees. Here, we provide this alternative formulation.

To use the symbolic factorization of Algorithm 8.1 for the structure of the matrix $A = M^TM$, we need to know the adjacent set of each node x_j in $G(A)$. But this is related to the structure of M by the next result.

PROPOSITION 8.2.

$$Adj_{G(A)}(x_j) = \{x_i \neq x_j \mid m_{rj} \neq 0, m_{ri} \neq 0 \ \textit{for some row}\ r\}. \ \square$$

In words, the structure of row/column j of A is given by the union of all the row structures M_{r*} of M where m_{rj} is nonzero. A direct application of Algorithm 8.1 to $Adj_{G(A)}(x_j)$ will determine the symbolic factorization of $A = M^TM$. But forming the entire structures of $Adj_{G(A)}(x_j)$ (for $j = 1, \cdots, n$) from M could be expensive.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In what follows, we shall show that for the purpose of performing the symbolic factorization of M^TM, we only need to form part of $Adj_{G(A)}(x_j)$. Define

$$FAdj_{G(A)}(x_j) = \{x_i \neq x_j \mid m_{r1} = \cdots = m_{r,j-1} = 0, m_{rj} \neq 0, m_{ri} \neq 0 \text{ for some row } r\}.$$

To simplify our notation, we shall use $Adj(x_j)$ and $FAdj(x_j)$ to refer to $Adj_{G(A)}(x_j)$ and $FAdj_{G(A)}(x_j)$, where $A = M^TM$. It follows from the definition that

$$FAdj(x_j) \subseteq Adj(x_j)\,.$$

However, the next theorem says that as far as adapting Algorithm 8.1 to the symbolic factorization of M^TM is concerned, it is sufficient to use $\{FAdj(x_j)\}$ instead of $\{Adj(x_j)\}$.

THEOREM 8.3. *For any node x_j in the elimination tree of $T(M^TM)$, $Adj(T[x_j])$ is given by*

$$FAdj(x_j) \cup \{\, Adj(T[x_s]) - \{x_j\} \mid x_s \text{ is a child of } x_j \,\}\,.$$

Proof. It follows from Theorem 8.1 that it is sufficient to show that

$$Adj(x_j) - \{x_1, \cdots, x_{j-1}\} \subseteq FAdj(x_j) \cup \{Adj(T[x_s]) - \{x_j\} \mid x_s \text{ is a child of } x_j\}.$$

Consider $x_i \in Adj(x_j)$ with $i > j$. If $x_i \in FAdj(x_j)$, there is nothing to prove. Otherwise, by Proposition 8.2, there exists a row r of M and a subscript $k < j$ such that

$$m_{rk} \neq 0\,,\ m_{rj} \neq 0\,,\ m_{ri} \neq 0\,.$$

This implies that a_{jk}, a_{ik}, and a_{ij} are nonzeros. By Theorem 3.1 on the elimination tree of $A = M^TM$, the node x_i is an ancestor of x_j, which in turn is an ancestor of x_k. Let x_s be the child of x_j, which is also an ancestor of x_k. Since $a_{ik} \neq 0$, by Theorem 3.5, $\ell_{is} \neq 0$. By Theorem 3.7, $x_i \in Adj(T[x_s]) - \{x_j\}$. □

The following algorithm performs the symbolic factorization scheme using the structure of M. Its correctness follows from Theorem 8.3.

Algorithm 8.2. Symbolic Factorization of $A = M^TM$.

```
for j := 1 to n do
begin
   Adj(T[x_j]) := ∅ ;
   for each row r of M with first nonzero in location j do
      Adj(T[x_j]) := Adj(T[x_j]) ∪ {x_i ≠ x_j | m_ri ≠ 0} ;
   for x_s in the children list of x_j do
      Adj(T[x_j]) := Adj(T[x_j]) ∪ Adj(T[x_s]) − {x_j} ;
   if Adj(T[x_j]) ≠ ∅ then
   begin
      p := min { i | x_i ∈ Adj(T[x_j]) } ;
      add x_j to the children list of x_p
   end
end.
```

Note that Algorithm 8.2 is essentially the same as Algorithm 8.1. The only difference is the use of the **for**-loop to initialize the structure of the factor column to

$FAdj(x_j)$ instead of $Adj(x_j)$ as in Algorithm 8.1. This formulation connects the algorithm by George and Ng [28] with the elimination tree structure and the original symmetric symbolic factorization scheme. The use of an extra integer vector $CLIST[*]$ to keep track of the children nodes during the algorithm can also be adapted from Algorithm 8.1.

9. Elimination trees in numerical factorization.

9.1. Sparse column-Cholesky factorization. Most implementations of sparse Cholesky factorization use some variants of the *column-Cholesky* scheme, whereby the sparse Cholesky factor is computed and accessed by columns. They include the Harwell MAxx series of routines for sparse solvers, the Waterloo SPARSPAK [24], and the Yale sparse matrix package [12]. Specifically, we can express the column scheme algorithmically as follows:

for $i := 1$ **to** n **do**
begin

$$\begin{pmatrix} t_i \\ \vdots \\ t_n \end{pmatrix} := \begin{pmatrix} a_{ii} \\ \vdots \\ a_{ni} \end{pmatrix} - \sum_{k<i} \ell_{ik} \begin{pmatrix} \ell_{ik} \\ \vdots \\ \ell_{nk} \end{pmatrix}$$

$$\begin{pmatrix} \ell_{ii} \\ \vdots \\ \ell_{ni} \end{pmatrix} := \frac{1}{\sqrt{t_i}} \begin{pmatrix} t_i \\ \vdots \\ t_n \end{pmatrix}$$

end

In this formulation, the temporary vector $(t_i, \cdots, t_n)^T$ is used here only for clarity. Its storage can overlap with that of $(\ell_{ii}, \cdots, \ell_{ni})^T$. This formulation is applicable to both dense and sparse matrices. In the sparse case, contributions to column i come from those preceding columns of L with nonzero ℓ_{ik}. These are given precisely by the row structure of L_{i*}, which is shown to be a subtree in the elimination tree. Recall from §3 that $T_r[x_i]$ is the row subtree, and it is a pruned subtree of $T[x_i]$ rooted at the node x_i. Although the elimination tree has no direct role in the sparse column-Cholesky approach, it does offer the numerical dependencies on columns of the Cholesky factor. The column dependency in Proposition 2.2 can be justified as follows.

PROPOSITION 9.1. *The column L_{*i} can be computed using A_{*i} and those columns L_{*k}, where $x_k \in T_r[x_i]$.* □

9.2. The multifrontal method for symmetric systems.

9.2.1. Frontal matrices and the elimination tree. The *multifrontal method* by Duff and Reid [10] is an important advance in direct solution of sparse systems. To factor a given large sparse matrix, the method uses a novel way of reorganizing the computation so that the entire sparse Cholesky factorization is performed through the partial factorization of a sequence of dense and smaller submatrices. The reorganization is based on the structure of the elimination tree (Duff and Reid use a slight variant, called an *assembly tree*). As before, let $x_1, x_2, \cdots, x_n$ be the ordering of the nodes in the graph $G(A)$. Recall that the node x_j corresponds to the jth row/column of the matrix A.

In the multifrontal method, each node x_j is associated with a *frontal matrix* $\bar{F}_j$, which is dense and much smaller. The size of the frontal matrix $\bar{F}_j$ is given by the

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

number of nonzeros in column j of L

$$\eta(L_{*j}) = |\, Adj(T[x_j]) \cup \{x_j\} \,|\,;$$

and its *first* row/column corresponds to the node x_j. Once fully formed, the first row/column in $\bar{F}_j$ can be eliminated from this matrix. Let F_j denote the remaining frontal matrix after the elimination of x_j from $\bar{F}_j$. We now describe the relation among these frontal matrices $\{\bar{F}_j\}$ and $\{F_j\}$ using the structure of the elimination tree.

For a given row/column j, consider the matrix B obtained by first removing all the rows and columns of A except those in $T[x_j] \cup Adj(T[x_j])$, and then zeroing out all the entries except those in the rows and columns of $T[x_j]$. For example, consider the matrix and elimination tree of Fig. 6.1. For node $c = x_4$, the corresponding B matrix is given by

$$\begin{pmatrix} a & \bullet & & \bullet \\ \bullet & c & \bullet & \\ & \bullet & 0 & 0 \\ \bullet & & 0 & 0 \end{pmatrix}$$

Note that the last two rows/columns correspond to the nodes g and h. Here, we use "0" to emphasize that zero value is assigned to the location under consideration. We provide the following observations without proof.

PROPOSITION 9.2. *The frontal matrix $\bar{F}_j$ is the same as the remaining matrix after the rows/columns of $T[x_j] - \{x_j\}$ have been eliminated from B.* □

PROPOSITION 9.3. *The matrix F_j is the same as the remaining matrix after the rows/columns of $T[x_j]$ have been eliminated from B.* □

From Proposition 9.3, the matrix F_j contains updates from the elimination of columns associated with $T[x_j]$. Moreover, these observations imply that the frontal matrix $\bar{F}_j$ can be assembled (or formed) using column A_{*j} of the given matrix and the remaining frontal matrices $\{F_s\}$ of its children nodes (if any) in the elimination tree. The algorithmic form of the multifrontal method can then be described in terms of the elimination tree as follows.

Algorithm 9.1. Multifrontal Method.

```
for j := 1 to n do
begin
   allocate space for the frontal matrix F̄_j ;
    for  each child x_s of x_j in the elimination tree T(A)   do
        assemble the remaining frontal matrix F_s into F̄_j ;
   assemble column A_{*j} into F̄_j ;
   perform one step of Cholesky factorization on F̄_j to give L_{*j} and F_j
end.
```

The assembly of the remaining frontal matrix F_s or column A_{*j} into $\bar{F}_j$ is performed by simply adding nonzero entries from F_s or A_{*j} into appropriate locations of $\bar{F}_j$. Note that the first row/column of the frontal matrix $\bar{F}_j$ corresponds to the node x_j, whose elimination from $\bar{F}_j$ gives the nonzero entries of L_{*j}. Therefore, the first column of the frontal matrix $\bar{F}_j$ is (logically) full. This implies that the remaining submatrix F_j is also full. This is perhaps the main characteristic of the multifrontal

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

method, which allows sparse factorization be performed via a sequence of full submatrices.

The multifrontal method was originally developed for the direct solution of symmetric indefinite sparse linear systems [10]. But it is already clear from this paper of Duff and Reid that the scheme can be used for sparse Cholesky factorization. In [51], Reid adapts this method in his TREESOLVE package, an out-of-core factorization scheme for large, sparse finite-element systems. Duff and Reid [9] demonstrate that the multifrontal method is especially effective on vector machines, such as the CRAY. Recently, Liu [41] showed that the method even has significant advantage over conventional sparse Cholesky schemes on virtual memory paging systems. The readers are referred to [2] and [3] for more recent development of the multifrontal method.

The role of the elimination tree in the multifrontal method should be clear from Algorithm 9.1. It provides the assembly connection among the frontal matrices. Indeed, each tree edge in the elimination tree corresponds to one assembly operation of some F_s into its parent's frontal matrix.

9.2.2. Matrix reordering for the multifrontal method. Implicitly assumed in Algorithm 9.1 is that in the assembly of the frontal matrix $\bar{F}_j$, the remaining frontal matrices $\{F_s\}$ of its children nodes are readily available. In other words, after performing one step of Cholesky factorization on $\bar{F}_j$, we need to store the remaining submatrix F_j for the assembly of its parent's frontal matrix. To reduce the amount of such storage, it is important to use F_j as early as possible to release its storage space for subsequent remaining frontal submatrices. This can be achieved by using a postordering of the elimination tree $T(A)$ of the matrix A. With a postordering, the logistics can be handled nicely by the use of a stack of remaining frontal matrices, as suggested by Duff and Reid [10]. Each F_s when required is always at the top of the stack. On the other hand, when F_j is formed, it is simply pushed into this stack. This stack of full triangular matrices (each F_j is symmetric) represents the working storage overhead for the implementation of the multifrontal method.

In [39], Liu provides an analysis of this working storage requirement in terms of the elimination tree structure. Based on the analysis, it is observed that the way in which children nodes are arranged can also affect the amount of working storage. But the reordering obtained from a rearrangement of children nodes still corresponds to a topological ordering of the elimination tree. By Theorem 6.1, it is an equivalent reordering (that is, it preserves the filled graph). Therefore, we are free to rearrange children nodes in the elimination tree.

Liu [39] shows that an optimal child sequence can be obtained by arranging the children nodes in descending order of some easily computed quantities. Significant reduction in working storage is achieved with a small investment in reordering time. This technique is recommended for any implementation of the multifrontal method.

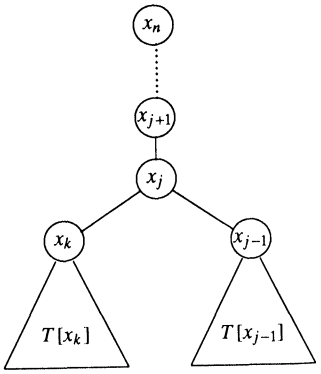
This storage reduction technique uses a reordering that preserves not only the filled graph, but also the elimination tree. If we remove the restriction on preserving the elimination tree, further working storage reduction is possible. In [43], Liu applies reorderings that restructure the elimination tree (as discussed in §6.3) to achieve more saving in this working storage overhead. The technique used will obtain an equivalent reordering (preserving the filled graph), that will give an "*unbalanced*" elimination tree. For the justification of this, we refer the reader to the paper. For an illustration, we note that the reordering given in Fig. 6.2 is a more desirable ordering than the one in Fig. 6.1 in terms of working storage requirement.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

9.3. Minimal storage sparse elimination algorithm. In reference [14], Eisenstat, Schultz, and Sherman propose an interesting variant of sparse Gaussian elimination for machines with limited core storage (without the use of auxiliary storage). The method trades an increase in computation for a decrease in storage by recomputing rather than saving most nonzeros in the triangular factor matrix. They have appropriately called it the *minimal storage sparse elimination* algorithm.

Central to their algorithm is a structure called the *element merge tree.* The tree structure can be interpreted as a variant of the elimination tree. We shall describe this minimal storage algorithm in terms of the elimination tree. Let A be the given n-by-n sparse symmetric positive-definite matrix and let $T(A)$ be its elimination tree. We assume that the ordering for A is already a postordering on the elimination tree $T(A)$.

Let x_j be the *last* node that has two or more children nodes. To simplify the exposition, we assume that x_j has two children nodes, one of them must be x_{j-1} (a property of a postordering). Let x_k be the other child. Pictorially, we can view that tree as follows:



The crucial observation in the scheme by Eisenstat, Schultz, and Sherman is that if solutions to the variables $\{x_j, \cdots, x_n\}$ are known, the problem can be reduced into two smaller independent systems, one involving variables in the subtree $T[x_k]$ and the other $T[x_{j-1}]$.

In matrix terms, we note that the given linear system can be partitioned as follows:

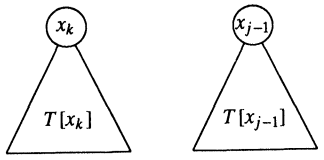
$$\begin{pmatrix} A_u & 0 & E_u^T \\ 0 & A_v & E_v^T \\ E_u & E_v & A_w \end{pmatrix} \begin{pmatrix} u \\ v \\ w \end{pmatrix} \begin{pmatrix} b_u \\ b_v \\ b_w \end{pmatrix},$$

where the solution vector x is partitioned into three parts u, v, and w. The subvector u corresponds to unknowns $x_1, \cdots, x_k$ in the subtree $T[x_k]$, v to $x_{k+1}, \cdots, x_{j-1}$ in $T[x_{j-1}]$, and w to $x_j, \cdots, x_n$. The matrix A and right-hand vector b are partitioned accordingly. The zero block submatrix in A follows from the fact that the two subtrees $T[x_k]$ and $T[x_{j-1}]$ are disjoint and Corollary 3.2. It is easy to see that if the solution w is known, the linear system is reduced to two smaller subsystems:

$$A_u\, u \;=\; b_u - E_u^T\, w\,, \quad A_v\, v \;=\; b_v - E_v^T\, w\,.$$

The elimination trees for the submatrices A_u and A_v are given by the subtrees $T[x_k]$ and $T[x_{j-1}]$, respectively. The scheme can then be interpreted in terms of the elimi-

nation tree as a removal of the nodes $\{x_j, \cdots, x_n\}$ from the elimination tree $T(A)$ to yield two trees for the independent subsystems:



Note that these two subsystems can be solved by applying the same approach recursively. The objective then becomes the determination of the variables $\{x_j, \cdots, x_n\}$ in w with limited core storage. The scheme by Eisenstat, Schultz, and Sherman can be best described using frontal matrices as described in §9.1. Indeed, [14] is perhaps the first paper that uses the multifrontal principle. We describe their overall scheme as follows.

Step 1. Determine the remaining frontal matrix F_k by eliminating nodes in the subtree $T[x_k]$.

Step 2. Determine the remaining frontal matrix F_{j-1} by eliminating nodes in the subtree $T[x_{j-1}]$.

Step 3. Subtract F_k and F_{j-1} from appropriate entries in A_w to give the modified submatrix $\bar{A}_w$. Let b_w be correspondingly modified to become $\bar{b}_w$. Then solve the linear subsystem $\bar{A}_w w = \bar{b}_w$.

Step 4. Solve $A_u u = b_u - E_u^T w$, and $A_v v = b_v - E_v^T w$.

The important point here is that during the formation of the frontal matrices F_k and F_{j-1}, the columns of L are discarded as they are computed. This saves storage for the factor matrix; and in exchange, part of columns of L have to be recomputed when solving for the unknowns in u and v.

9.4. Row merging scheme for orthogonal decomposition. In [38], Liu introduces the notion of a *row merge tree* for sparse orthogonal decomposition by Givens rotations. In that paper, the connection between a row merge tree and an elimination tree is addressed, and the relation of row merging for sparse QR factorization with the multifrontal method for sparse Cholesky factorization is also briefly mentioned. Here, we give a more detailed account of the relation among elimination tree, row merging, and the multifrontal method.

Let M be an m-by-n large sparse matrix with full column rank. The problem is to determine the triangular factor R in the orthogonal decomposition of M into

$$Q \begin{pmatrix} R \\ 0 \end{pmatrix},$$

where Q is an m-by-m orthogonal matrix. It is well known that the factor R is mathematically equivalent to the Cholesky factor of the symmetric matrix $A = M^T M$.

Assume that the columns of M have been prearranged by some fill-reducing ordering on $A = M^T M$ (as suggested by George and Heath [20]). Let $T(A)$ be the elimination tree of A. The general row merging scheme [38] will organize computation for the sparse orthogonal decomposition using the structure of the elimination tree. The way computation is organized is similar to the multifrontal method for sparse Cholesky factorization. Indeed, it is appropriate to view the general row merging scheme as one that performs the multifrontal method on $A = M^T M$ without ever forming A. Instead, the given matrix M is used.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In the multifrontal method (Algorithm 9.1), each node x_j is associated with a frontal matrix $\bar{F}_j$, and a logically full submatrix F_j. At step j, the numerical computation consists of the formation of the frontal matrix $\bar{F}_j$ through assembly, and the elimination of a variable from this frontal matrix to give F_j.

In the case of the general row merging scheme for the given matrix M, the computation associated with each node x_j of the elimination tree $T(M^TM)$ is a merging process involving a set of full upper trapezoidal submatrices together with some original rows of the matrix M. The original rows from M are those with the first nonzero in location j; while each child node of x_j will contribute one such upper trapezoidal submatrix to the merging operation. The merging operation uses a sequence of Givens rotations to reduce the set of upper trapezoidal submatrices and selected rows from M, to form another full upper trapezoidal submatrix, part of which will in turn be passed to the parent node of x_j.

Let us denote by $\bar{Z}_j$ the full upper trapezoidal submatrix for node x_j and by Z_j the remaining submatrix after the removal of the first row from $\bar{Z}_j$. We state without proof the following observations on $\bar{Z}_j$ and Z_j.

PROPOSITION 9.4. *The size of the submatrix $\bar{Z}_j$ $\{Z_j\}$ is the same as that of $\bar{F}_j$ $\{F_j\}$.* □

PROPOSITION 9.5. *If QR is the orthogonal decomposition of the matrix M, then the jth row of the upper triangular factor matrix is given by the first row of $\bar{Z}_j$.* □

The general row merging scheme can be described in terms of the submatrices $\bar{Z}_j$ and Z_j. Note the similar logic structure as in the multifrontal method of Algorithm 9.1.

Algorithm 9.2. General Row Merging Scheme.

```
for j := 1 to n do
begin
   allocate space for the upper trapezoidal submatrix Z̄_j;
   for each child x_s of x_j in the elimination tree T(M^T M) do
      merge the remaining trapezoidal submatrix Z_s into Z̄_j;
   for each row r of M with first nonzero at location j do
      merge row M_{r *} into Z̄_j;
   remove the first row of Z̄_j to give the jth row of R and Z_j;
end.
```

The similarity between Algorithm 9.2 and Algorithm 9.1 is clear. Essentially, the "assemble" operation is replaced by the "merge" operation. It is also interesting to point out that Algorithm 9.2 can be viewed simply as an extension of the symbolic factorization scheme for $A = M^TM$ in Algorithm 8.2. Logical manipulation of the structure of M is now replaced by the actual numerical computation involved in forming rows of the upper triangular factor R. In this paper, we are concerned with the exposition of the connection between the general row merging scheme and the elimination tree. For details of the merging scheme, the reader is referred to [23] and [38].

9.5. Factorization of symmetric indefinite systems. In [10], Duff and Reid introduce the multifrontal method as an efficient scheme to solve sparse symmetric indefinite systems. Let A be an n-by-n symmetric indefinite matrix. The elimination tree structure $T(A)$ is still defined. Assume that the ordering on A is already a postordering on the tree $T(A)$.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

If the matrix A were positive definite, the multifrontal method would form the frontal matrix $\bar{F}_j$ at step j and proceed to eliminate variable x_j from this frontal matrix. However, since A is now indefinite, the variable x_j may not be a suitable numerical pivot in the frontal matrix $\bar{F}_j$ (though it is a good structural pivot). In such case, Duff and Reid allow the frontal matrix $\bar{F}_j$ to be extended to include more rows/columns, thereby enlarging the set of pivot candidates.

As noted by Liu [44], the scheme of Duff and Reid uses the technique of *delayed elimination*. When a node is deemed as unsuitable to be a numerical pivot, its elimination will be delayed to a later stage. This is different from the conventional approach for factoring dense indefinite matrices, where *advanced* elimination is used to eliminate the suitable node as part of a 2×2 block pivot [5].

The connection between the elimination tree and delayed elimination is discussed in [44]. The elimination tree represents a class of ideal elimination sequences in which no pivoting is performed. With the added stability requirement, we can still use the elimination tree structure to provide a selection guide for 1×1 and 2×2 pivots. At step j, pivots can only be selected from the uneliminated nodes of the subtree $T[x_j]$. If no satisfactory pivots can be found, the scheme will proceed to work on the next node x_{j+1}. On the other hand, if uneliminated nodes from this subtree are selected as pivots, the amount of structural damage to the resulting tree will be contained. In this way, the resulting sequence of stable pivots will form an elimination tree that deviates as little as possible from the original tree. We shall refer the interested readers to [44] for more details.

10. Elimination trees in computing environments.

10.1. Factorization on a paging environment. In §6, we consider equivalent matrix reorderings on a given sparse matrix A using the structure of its elimination tree $T(A)$. They all preserve the structure of the filled graph of A, and will require the same amount of storage and computational cost in factoring A. Those based on topological orderings on the tree $T(A)$ will also preserve the elimination tree structure as considered in §6.1. There are also some that will restructure the tree, as given by those from rotations in §6.2.

For a given computing environment, a meaningful question to ask is to find the "best" equivalent reordering for the factorization of the sparse matrix A. Even finding a "good" equivalent reordering for a particular environment will be of practical significance. In this section, we consider some common computing environments and offer some suggestions for obtaining "good" reorderings. The recommendations are based on intuitive understanding of the environment and the factorization process, and are further substantiated by numerical experiments. Further work is still needed for theoretical justification of these recommendations. The recent work by Pothen [49] is on the complexity of finding optimal elimination trees.

We first consider the sparse Cholesky factorization on a virtual memory paging system. In this environment, it is desirable to have columns of the matrix organized so as to reduce the amount of paging activity. This obviously depends on the numerical factorization algorithm used. In [42], Liu considers the conventional sparse column-Cholesky factorization scheme as discussed in §9.1.

From Proposition 9.1, in the factorization scheme by columns, each column of the factor matrix is computed by a number of column modifications, governed by the locations of the nonzeros in its corresponding row. Each row structure is itself a (pruned) subtree of the elimination tree $T(A)$. Therefore, a reasonable recommendation is to order nodes in each subtree consecutively, so that columns for each subtree

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

will be stored in contiguous memory locations. In this way, columns required for the computation of L_{*j} will reside relatively close to column j. This tends to improve on the *locality* of the factorization scheme.

A postordering on the elimination tree is an appropriate choice. It is demonstrated that for some practical problems, the amount of CPU time for factorization can differ by almost 100 percent for two equivalent reorderings [42].

10.2. Out-of-core sparse factorization. In the situation when the storage requirement of the matrix factor L of A exceeds the space available in main memory, it is necessary to exploit the use of auxiliary storage. The multifrontal method as described in Algorithm 9.1 lends itself readily to such an environment. After performing one step of Cholesky factorization on the frontal matrix $\bar{F}_j$ to give L_{*j} and F_j, the out-of-core version will store the factor column L_{*j} into secondary storage. This will not affect subsequent steps of the factorization, since this column L_{*j} is no longer necessary. Indeed, contributions from it have already been accumulated in the remaining frontal matrix F_j.

The amount of primary storage to perform the entire factorization of this out-of-core multifrontal method is precisely the same as the working storage requirement for the in-core version. Therefore, the use of tree rotations to give an "unbalanced" tree structure and the technique of optimal children resequencing on the resulting elimination tree can also be applied here to reduce the amount of storage for the out-of-core scheme. The relevance of the elimination tree is clear for this situation.

In [40], Liu proposes an alternative general sparse out-of-core scheme, which is based on reorganization of the data storage vector during the course of sparse factorization by columns. Column segments of the factor matrix that are no longer needed will be discarded to make room for new columns. The elimination tree is again a relevant structure in determining the primary storage requirement. The techniques of unbalanced tree and optimal children resequencing are also applicable here, although the formula for storage requirement is slightly different. Interested readers can consult [40] for the explicit formula.

10.3. Models for parallel factorization. Parallel machines offer a different computing environment. The use of multiprocessors in the parallel Cholesky factorization of large sparse matrices has attracted the attention of many researchers. Jess and Kees [34] use the elimination tree structure as a large-grained task model for sparse factorization. In [37], Liu presents a systematic and unified treatment of computational task models for parallel sparse factorization. It is shown that the elimination tree is an appropriate large grained model irrespective of whether computation is performed by row, by column or by submatrix. The next result follows directly from Proposition 9.1.

PROPOSITION 10.1. *Let $T[x_i]$ and $T[x_j]$ be two disjoint subtrees of the elimination tree $T(A)$. The columns L_{*i} and L_{*j} can be computed in parallel with no overlap in data access.* □

COROLLARY 10.2. *All the leaf nodes in the elimination tree can be eliminated in parallel.* □

Nodes that can be eliminated in parallel are called *parallel pivots.* In this context, it is desirable to maximize the number of parallel pivots for each step without sacrificing the fill-reducing quality of the ordering. Here, each step refers to the elimination of a set of parallel pivots. The approach that Jess and Kees [34] take is to consider *equivalent* matrix reorderings that will maximize the number of parallel pivots. They provide an algorithm to determine such an equivalent ordering using the filled graph

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

of the matrix. In [47], a linear algorithm is given that achieves the same result. Furthermore, it is shown in [46] that the resulting elimination tree has minimum height among all trees from the class of equivalent reorderings. Recent works by Lewis and Peyton [35] and Pothen [50] provide further insight into the algorithm by Jess and Kees and contain improvement to [47].

Another recent use of elimination tree in the context of parallel factorization is in the assignment of computational tasks to individual processors in a multiprocessor environment. For shared memory architectures, Liu [37] shows how the structure of the elimination tree can be used to schedule computational tasks to the processors. In the case of local memory systems, it is desirable to assign tasks to processors to achieve load balancing and to reduce communication traffic between processors. In [25], George, Liu, and Ng show how an assignment of tasks to processor nodes of a hypercube architecture can be performed to satisfy these objectives. The key idea used is to map tasks associated with a subtree of the elimination tree to a subcube of the hypercube.

Duff [7] considers the parallel version of the multifrontal method. The way in which elimination can be performed in multiple independent fronts is governed by the structure of the elimination tree. Other recent works on parallel sparse factorization that involve the elimination tree include George et al [19], Gilbert and Hafsteinsson [29], and Zmijewski [61].

11. Other related notions and future research directions. It is quite evident that the elimination tree is a valuable structure in the study of sparse matrix factorization. Our exploration of its usage in this paper is by no means exhaustive. Some useful block partitionings have been defined based on the elimination tree structure. One of them has been referred to as a *supernodal partitioning* [2], [3]. In its simplest form, a supernodal partitioning can be defined as follows. Let x_p be the parent node of x_j in the elimination tree. These two nodes x_j and x_p belong to the same block in the partitioning if

$$Adj(T[x_j]) = Adj(T[x_p]) \cup \{x_p\},$$

and x_j is the only child of x_p. (Note that by Theorem 7.1 the above condition is equivalent to $| Adj(T[x_j]) | = | Adj(T[x_p]) | + 1$.) The set of nodes in each block is collectively referred to as a *supernode*. It can be readily verified that each supernode defines a clique in the filled graph and they share the same set of adjacent nodes outside the clique. Furthermore, the nodes of each supernode corresponds to a chain in the elimination tree. The notion of supernodal partitioning plays a central role in devising highly efficient multifrontal codes on vector machines [2], [3].

The compressed column storage scheme by Sherman [56] can also be viewed as using a generalization of this notion of supernodes. If we number the nodes of each supernode consecutively, each supernode corresponds to a full diagonal block of the Cholesky factor. Since the columns associated with each supernode have identical column structures outside the diagonal block, we need to store only one copy of the column structures for each supernode. The success of Sherman's compressed column storage scheme can be attributed mostly to this property of supernodes.

In his thesis [48], Peters considers a different partitioning based on the elimination tree structure. If x_p is the parent node of x_j, these two nodes belong to the same partition if x_j is the only child of x_p. He refers to it as a *proper perfect preserving partition*. His motivation is to use the partition to devise a solution method that only requires full envelope (or profile) solves.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

In this paper, we have not discussed the notion of *clique trees* [35] defined for chordal graphs (or filled graphs). A clique tree has a structure closely related to that of an elimination tree. An application of it to sparse matrix reordering for parallel elimination can be found in [35].

We have discussed some of the roles of elimination trees in sparse QR and LU factorizations in §§7.3, 8.3, 9.4, and 9.5. The direct solution of unsymmetric sparse linear systems is a relatively less developed area. With more advances in sparse QR and LU algorithms for unsymmetric systems in the future, we expect to see more impact from this tree structure.

The elimination tree structure provides information on data dependency in the factorization process. It captures the essential ingredient for parallel elimination. With the explosion in research work on parallel algorithms, this tree will definitely play a central role in future development. The height of the elimination tree represents an effective but crude measure for the amount of work in parallel elimination. Finding practical and more refined criteria in terms of the tree will be a fruitful research area.

Another important research direction is in the characterizations of the best elimination tree structure for a given computational environment, be it on a parallel architecture, a vector machine, or a virtual memory system. Current recommendations on desirable structures are mostly based on intuition and experience. A vigorous approach to provide theoretical justification seems to be an interesting and important area.

Acknowledgments. The author would like to thank John Gilbert and the two referees for their careful and critical reading of the original manuscript.

REFERENCES

[1] A. V. AHO, J. E. HOPCROFT, AND J. D. ULLMAN, *Data Structures and Algorithms*, Addison-Wesley, Reading, MA, 1983.

[2] C. ASHCRAFT, *A vector implementation of the multifrontal method for large sparse, symmetric positive definite linear systems*, Tech. Report ETA-TR-51, Engineering Technology Applications Division, Boeing Computer Services, Seattle, WA, 1987.

[3] C. ASHCRAFT AND R. GRIMES, *The influence of relaxed supernode partitions on the multifrontal method*, Tech. Report ETA-TR-60, Engineering Technology Applications Division, Boeing Computer Services, Seattle, WA, 1987.

[4] R. E. BANK AND R. K. SMITH, *General sparse elimination requires no permanent integer storage*, SIAM J. Sci. Statist. Comput., 8 (1987), pp. 574–584.

[5] J. R. BUNCH AND L. KAUFMAN, *Some stable methods for calculating inertia and solving symmetric linear equations*, Math. Comp., 31 (1977), pp. 163–179.

[6] I. S. DUFF, *Full matrix techniques in sparse Gaussian elimination*, in Lecture Notes in Mathematics (912), G. Watson, ed., Springer-Verlag, Berlin, New York, 1982, pp. 71–84.

[7] ———, *Parallel implementation of multifrontal schemes*, Parallel Comput., 3 (1986), pp. 193–204.

[8] I. S. DUFF, A. M. ERISMAN, AND J. K. REID, *Direct Methods for Sparse Matrices*, Oxford University Press, London, 1987.

[9] I. S. DUFF AND J. K. REID, *Experience of sparse matrix codes on the CRAY-1*, Comput. Phys. Comm., 26 (1982), pp. 293–302.

[10] ———, *The multifrontal solution of indefinite sparse symmetric linear equations*, ACM Trans. Math. Software, 9 (1983), pp. 302–325.

[11] ———, *A note on the work involved in no-fill sparse matrix factorization*, IMA J. Numer. Anal., 3 (1983), pp. 37–40.

[12] S. C. EISENSTAT, M. C. GURSKY, M. H. SCHULTZ, AND A. H. SHERMAN, *The Yale sparse matrix package, 1. The symmetric code*, Internat. J. Numer. Meth. Engrg., 18 (1982), pp. 1145–1151.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

[13] S. C. EISENSTAT, M. H. SCHULTZ, AND A. H. SHERMAN, *Applications of an element model for Gaussian elimination*, in Sparse Matrix Computations, J. R. Bunch and D. J. Rose, eds., Academic Press, New York, 1976, pp. 85–96.

[14] ———, *Software for sparse Gaussian elimination with limited core storage*, in Sparse Matrix Proceedings, I. S. Duff and G. W. Stewart, eds., Society for Industrial and Applied Mathematics, Philadelphia, PA, 1979, pp. 135–153.

[15] ———, *Algorithms and data structures for sparse symmetric Gaussian elimination*, SIAM J. Sci. Statist. Comput., 2 (1981), pp. 225–237.

[16] M. J. FISCHER, *Efficiency of equivalence algorithms*, in Complexity of Computer Computations, R. E. Miller and J. W. Thatcher, eds., Plenum Press, New York, 1972, pp. 153–167.

[17] F. GAVRIL, *The intersection graphs of subtrees in trees are exactly the chordal graphs*, J. Combin. Theory Ser. B, 16 (1974), pp. 47–56.

[18] J. A. GEORGE, *Nested dissection of a regular finite element mesh*, SIAM J. Numer. Anal., 10 (1973), pp. 345–363.

[19] J. A. GEORGE, M. HEATH, J. W. H. LIU, AND E. NG, *Sparse Cholesky factorization on a local-memory multiprocessor*, SIAM J. Sci. Statist. Comput., 9 (1988), pp. 327–340.

[20] J. A. GEORGE AND M. T. HEATH, *Solution of sparse linear least squares problems using Givens rotations*, Linear Algebra Appl., 34 (1980), pp. 69–83.

[21] J. A. GEORGE AND J. W. H. LIU, *An optimal algorithm for symbolic factorization of symmetric matrices*, SIAM J. Comput., 9 (1980), pp. 583–593.

[22] ———, *Computer Solution of Large Sparse Positive Definite Systems*, Prentice-Hall, Englewood Cliffs, NJ, 1981.

[23] ———, *Householder reflection versus Givens rotations in sparse orthogonal decomposition*, Linear Algebra Appl., 88/89 (1987), pp. 223–238.

[24] J. A. GEORGE, J. W. H. LIU, AND E. NG, *User guide for SPARSPAK, Waterloo Sparse Linear Equations Package*, Tech. Report Res. CS-78-30 (revised 1980), Department of Computer Science, University of Waterloo, Waterloo, Ontario, Canada, 1980.

[25] ———, *Communication reduction in parallel sparse Cholesky factorization on a hypercube*, in Hypercube Multiprocessors 1987, M. T. Heath, ed., Society for Industrial and Applied Mathematics, Philadelphia, PA, 1987, pp. 576–586.

[26] ———, *A data structure for sparse QR and LU factorization*, SIAM J. Sci. Statist. Comput., 9 (1988), pp. 100–121.

[27] J. A. GEORGE AND E. NG, *An implementation of Gaussian elimination with partial pivoting for sparse systems*, SIAM J. Sci. Statist. Comput., 6 (1985), pp. 390–409.

[28] ———, *Symbolic factorization for sparse Gaussian elimination with partial pivoting*, SIAM J. Sci. Statist. Comput., 8 (1987), pp. 877–898.

[29] J. R. GILBERT AND H. HAFSTEINSSON, *A parallel algorithm for finding fill in a sparse symmetric matrix*, Tech. Report TR 86-789, Department of Computer Science, Cornell University, Ithaca, NY, 1986.

[30] J. R. GILBERT, D. J. ROSE, AND A. EDENBRANDT, *A separator theorem for chordal graphs*, SIAM J. Algebraic Discrete Methods, 5 (1984), pp. 306–313.

[31] J. R. GILBERT AND R. E. TARJAN, *The analysis of a nested dissection algorithm*, Numer. Math., 50 (1987), pp. 377–404.

[32] M. C. GOLUMBIC, *Algorithmic Graph Theory and Perfect Graphs*, Academic Press, New York, 1980.

[33] J. E. HOPCROFT AND R. E. TARJAN, *Dividing a graph into triconnected components*, SIAM J. Comput., 2 (1973), pp. 135–158.

[34] J. A. G. JESS AND H. G. M. KEES, *A data structure for parallel L/U decomposition*, IEEE Trans. Comput., C-31 (1982), pp. 231–239.

[35] J. G. LEWIS AND B. W. PEYTON, *A fast implementation of the Jess and Kees algorithm*, Tech. Report ETA-TR-90, Engineering and Scientific Services Division, Boeing Computer Services, Seattle, WA, 1988.

[36] J. W. H. LIU, *A compact row storage scheme for Cholesky factors using elimination trees*, ACM Trans. Math. Software, 12 (1986), pp. 127–148.

[37] ———, *Computational models and task scheduling for parallel sparse Cholesky factorization*, Parallel Comput., 3 (1986), pp. 327–342.

[38] ———, *On general row merging schemes for sparse Givens transformations*, SIAM J. Sci. Statist. Comput., 7 (1986), pp. 1190–1211.

[39] ———, *On the storage requirement in the out-of-core multifrontal method for sparse factorization*, ACM Trans. Math. Software, 12 (1986), pp. 249–264.

[40] ———, *An adaptive general sparse out-of-core Cholesky factorization scheme*, SIAM J. Sci. Statist. Comput., 8 (1987), pp. 585–599.

[41] ———, *The multifrontal method and paging in sparse Cholesky factorization*, Tech. Report CS-87-09, Department of Computer Science, York University, North York, Ontario, Canada, 1987.

[42] ———, *A note on sparse factorization in a paging environment*, SIAM J. Sci. Statist. Comput., 8 (1987), pp. 1085–1088.

[43] ———, *Equivalent sparse matrix reordering by elimination tree rotations*, SIAM J. Sci. Statist. Comput., 9 (1988), pp. 424–444.

[44] ———, *A tree model for sparse symmetric indefinite matrix factorization*, SIAM J. Matrix Anal. Appl., 9 (1988), pp. 26–39.

[45] ———, *A graph partitioning algorithm by node separators*, ACM Trans. Math. Software, 15 (1989), pp. 198–219.

[46] ———, *Reordering sparse matrices for parallel elimination*, Parallel Comput., 11 (1989), pp. 73–91.

[47] J. W. H. LIU AND A. MIRZAIAN, *A linear reordering algorithm for parallel pivoting of chordal graphs*, SIAM J. Discrete Math., 2 (1989), pp. 100–107.

[48] F. J. PETERS, *Sparse matrices and substructures*, Tech. Report Mathematical Centre Tracts 119, Matematisch Centrum, Amsterdam, the Netherlands, 1979.

[49] A. POTHEN, *The complexity of optimal elimination trees*, Tech. Report CS-88-16, Department of Computer Science, The Pennsylvania State University, University Park, PA, 1988.

[50] ———, *Simplicial cliques, shortest elimination trees, and supernodes in sparse cholesky factorization*, Tech. Report CS-88-13, Department of Computer Science, The Pennsylvania State University, University Park, PA, 1988.

[51] J. K. REID, TREESOLVE, *a Fortran package for solving large sets of linear finite element equations*, Tech. Report CSS 155, Computer Sciences and Systems Division, AERE Harwell, Oxfordshire, 1984.

[52] D. J. ROSE, *A graph-theoretic study of the numerical solution of sparse positive definite systems of linear equations*, in Graph Theory and Computing, R. Read, ed., Academic Press, New York, 1972, pp. 183–217.

[53] D. J. ROSE, R. E. TARJAN, AND G. S. LUEKER, *Algorithmic aspects of vertex elimination on graphs*, SIAM J. Comput., 5 (1976), pp. 266–283.

[54] D. J. ROSE, G. F. WHITTEN, A. H. SHERMAN, AND R. E. TARJAN, *Algorithms and software for in-core factorization of sparse symmetric positive definite matrices*, Computers & Structures, 11 (1980), pp. 597–608.

[55] R. SCHREIBER, *A new implementation of sparse Gaussian elimination*, ACM Trans. Math. Software, 8 (1982), pp. 256–276.

[56] A. H. SHERMAN, *On the efficient solution of sparse systems of linear and nonlinear equations*, PhD thesis, Department of Computer Science, Yale University, New Haven, CT, 1975.

[57] R. E. TARJAN, *Depth-first search and linear graph algorithms*, SIAM J. Comput., 1 (1972), pp. 146–160.

[58] ———, *Data Structures and Network Algorithms*, CBMS–NSF Regional Conference Series in Applied Mathematics 44, Society for Industrial and Applied Mathematics, Philadelphia, PA, 1983.

[59] R. E. TARJAN AND M. YANNAKAKIS, *Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs*, SIAM J. Comput., 13 (1984), pp. 566–579.

[60] G. F. WHITTEN, *Computation of fill-in for sparse symmetric positive definite matrices*, Tech. Report Unpublished manuscript, Computer Science Department, Vanderbilt University, Nashville, TN, 1978.

[61] E. ZMIJEWSKI, *Sparse Cholesky factorization on a multiprocessor*, PhD thesis, Department of Computer Science, Cornell University, Ithaca, NY, 1987.

[62] E. ZMIJEWSKI AND J. R. GILBERT, *A parallel algorithm for large sparse Cholesky factorization on a multiprocessor*, Tech. Report TR 86-733, Department of Computer Science, Cornell University, Ithaca, NY, 1986.

Downloaded 01/04/13 to 128.148.252.35. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php